Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase II · Aprendizaje automático y redes neuronales

Módulo 11

Entrenar redes profundas

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Un modelo de lenguaje actual apila decenas de bloques idénticos y se entrena en aritmética de 16 bits durante billones de tokens. Las dos circunstancias son hostiles al descenso de gradiente. La profundidad convierte el gradiente de las primeras capas en un producto de muchas matrices jacobianas, cuya norma tiende a crecer o a decrecer exponencialmente; la precisión reducida hace que sumas, exponenciales y cuadrados desborden, se anulen o pierdan los sumandos pequeños. Este módulo estudia, con el mismo grado de formalización que los anteriores, las piezas que hacen posible el entrenamiento a pesar de ello: la inicialización calibrada, las conexiones residuales, la normalización por capa en sus variantes LayerNorm y RMSNorm, la posición pre-norm de la normalización, el dropout, el recorte del gradiente por norma global, la aritmética de coma flotante con la precisión mixta y la evaluación estable de la función log-sum-exp. Todas ellas aparecen en el bloque del Transformer que se construye en la fase III.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. expresar el gradiente de una red profunda como producto de jacobianos, acotar su norma mediante valores singulares y explicar el desvanecimiento y la explosión del gradiente, incluida la condición de Pascanu, Mikolov y Bengio para redes recurrentes;

2. demostrar la ley de propagación de la varianza y deducir de ella las inicializaciones de Xavier y de He;

3. demostrar que el jacobiano de una red residual contiene la identidad, acotar inferiormente su menor valor singular y describir el flujo residual de un Transformer;

4. calcular LayerNorm y RMSNorm, demostrar sus invariancias y la ortogonalidad del gradiente que atraviesa RMSNorm, y justificar la posición pre-norm en los modelos de lenguaje;

5. demostrar que el dropout invertido es insesgado, calcular la varianza que introduce y aplicar el recorte de gradiente por norma global;

6. deducir el rango, la precisión y la unidad de redondeo de fp32, fp16, bf16 y fp8 a partir de sus bits, explicar la absorción de sumandos y los tres ingredientes de la precisión mixta;

7. evaluar log-sum-exp, softmax y entropía cruzada de forma estable y diagnosticar el origen de una pérdida que se vuelve `NaN`.

Requisitos previos

Del módulo 3, los valores singulares, la norma espectral y la descomposición espectral de una matriz simétrica (temas 1 y 2). Del módulo 4, la regla de la cadena para funciones vectoriales y la matriz jacobiana, que se usan en todos los temas. Del módulo 5, esperanza, varianza, independencia y la distribución de Bernoulli (temas 1, 2 y 4). Del módulo 7, el descenso de gradiente estocástico, Adam y el calentamiento de la tasa de aprendizaje (temas 3 y 4). Del módulo 9, el perceptrón multicapa y las funciones de activación; del módulo 10, la retropropagación en grafos computacionales. La softmax y la entropía cruzada del módulo 1 se retoman en el tema 6.

Organización

Tema	Contenido	Fuentes principales
1	Gradientes que se desvanecen o explotan	[9], [26], [18], [7]
2	Conexiones residuales	[10], [12], [26], [2]
3	Normalización: LayerNorm, RMSNorm y pre-norm	[1], [27], [25], [2]
4	Dropout y recorte del gradiente	[21], [9], [18], [3]
5	Aritmética de coma flotante y precisión mixta	[14], [8], [16], [5]
6	Estabilidad numérica de softmax y entropía cruzada	[9], [26], [5]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. El análisis del gradiente en profundidad sigue la sección 8.2.5 y el capítulo 10 (§10.7 y §10.11) de Goodfellow, Bengio y Courville [9], las secciones 5.4 y 9.7 de Zhang *et al.* [26] y el artículo de Pascanu, Mikolov y Bengio [18]; la inicialización, los artículos de Glorot y Bengio [7] y de He *et al.* [11]. Las conexiones residuales y la normalización siguen la sección 8.6 de [26] y las secciones 7.4 y 9.5 de Bishop y Bishop [2], junto con los artículos originales de He *et al.* [10, 12], Ba, Kiros y Hinton [1], Zhang y Sennrich [27] y Xiong *et al.* [25]. El dropout sigue la sección 7.12 de [9], la sección 5.6 de [26], las notas de CS231n [3] y el artículo de Srivastava *et al.* [21]. La aritmética de coma flotante sigue la norma IEEE 754 [14], el tutorial de Goldberg [8] y el libro de Higham [13]; la precisión mixta, los artículos de Micikevicius *et al.* [16, 17]. Varios ejercicios proceden de las prácticas del curso CS336 de Stanford [5, 6] y del curso CS224N [4]; el curso 11-785 de CMU [20] cubre el mismo temario. Sobre Claude, Gemini y Grok solo se usa información pública: Anthropic y Google no documentan la normalización ni la precisión con que entrenan sus modelos; el código publicado de Grok-1 [24] sí permite consultarlas.

Notación

Símbolo	Significado
$h_\ell \in \mathbb{R}^d$, $z_\ell = W_\ell h_{\ell-1}$	activación y preactivación de la capa ℓ, con $\ell = 1, \dots, L$
$J_\ell = \partial h_\ell / \partial h_{\ell-1}$	jacobiano de la capa (o de la rama residual, tema 2)
$\mathcal{L}$, $g = \nabla_\theta \mathcal{L}$	pérdida y gradiente respecto de los parámetros
$\|A\|_2$, $\sigma_{\min}(A)$	norma espectral (mayor valor singular) y menor valor singular
Var, $\mathbb{E}$	varianza y esperanza
μ, σ^2, $\mathrm{RMS}(x)$	media, varianza y raíz cuadrática media de un vector
γ, β; g	ganancia y sesgo de LayerNorm; ganancia de RMSNorm
ϵ	constante de estabilidad dentro de una raíz
p, $m_i \sim \mathrm{Bernoulli}(1-p)$	probabilidad de apagado del dropout y máscara
c	umbral del recorte del gradiente
e, m	bits de exponente y de mantisa de un formato de coma flotante
$\varepsilon = 2^{-m}$, $u = \varepsilon/2$	épsilon de máquina y unidad de redondeo
$\mathrm{fl}(x)$	número representable más cercano a x
$\mathrm{LSE}(z)$	log-sum-exp, $\ln \sum_i e^{z_i}$
$u \odot w$, $\mathbf{1}$	producto elemento a elemento; vector de unos

Índice

Presentación del módulo **ii**

1 Gradientes que se desvanecen o explotan **1**

1.1 El gradiente como producto de jacobianos 1

1.2 Cotas mediante valores singulares 2

1.3 Saturación de las activaciones 3

1.4 Propagación de la varianza e inicialización 3

Ejercicios del tema 1 5

2 Conexiones residuales **6**

2.1 Clases de funciones anidadas 6

2.2 Propagación en una red residual 6

2.3 El flujo residual de un Transformer 8

Ejercicios del tema 2 8

3 Normalización: LayerNorm, RMSNorm y pre-norm **10**

3.1 Normalización por lotes y por capa 10

3.2 Invariancias 11

3.3 Post-norm y pre-norm 12

Ejercicios del tema 3 13

4 Dropout y recorte del gradiente **14**

4.1 Dropout 14

4.2 Recorte del gradiente por norma global 15

Ejercicios del tema 4 16

5 Aritmética de coma flotante y precisión mixta **18**

5.1 Formatos de coma flotante 18

5.2 Redondeo y unidad de redondeo 19

5.3 Precisión mixta 20

Ejercicios del tema 5 21

6 Estabilidad numérica de softmax y entropía cruzada **23**

6.1 Límites de la exponencial 23

6.2 La función log-sum-exp 23

6.3 Entropía cruzada a partir de los logits 24

6.4 Propagación de `NaN` y diagnóstico 25

Ejercicios del tema 6 26

A Soluciones de los ejercicios **27**

A.1 Tema 1 27

A.2 Tema 2 29

A.3 Tema 3 31

A.4 Tema 4 34

A.5 Tema 5 37

A.6 Tema 6 40

Bibliografía **43**

Tema 1

Gradientes que se desvanecen o explotan

En una red profunda, el gradiente que recibe una capa es el producto de los jacobianos de todas las capas que tiene por encima. Este tema formaliza esa observación, acota la norma del producto mediante valores singulares y deduce de ella los dos fenómenos que dificultan el entrenamiento en profundidad: el desvanecimiento y la explosión del gradiente. Se estudian después sus dos causas principales, la escala de los pesos y la saturación de las activaciones, y se obtiene la inicialización que mantiene constante la varianza de la señal. Se sigue [9, §8.2.5, §10.7], [26, §5.4 y §9.7] y los artículos de Pascanu, Mikolov y Bengio [18], Glorot y Bengio [7] y He *et al.* [11].

1.1 El gradiente como producto de jacobianos

Definición 1.1 (Red profunda de anchura constante). Sea $L \geq 1$. Una *red de L capas* con anchura d y activación $f\colon \mathbb{R} \to \mathbb{R}$ derivable transforma una entrada $h_0 \in \mathbb{R}^d$ mediante

$$z_\ell = W_\ell h_{\ell-1}, \qquad h_\ell = f(z_\ell), \qquad \ell = 1, \dots, L,$$

con $W_\ell \in \mathbb{R}^{d\times d}$ y f aplicada componente a componente. La pérdida $\mathcal{L}$ depende de la entrada solo a través de h_L. El *jacobiano de la capa ℓ* es

$$J_\ell = \frac{\partial h_\ell}{\partial h_{\ell-1}} = \operatorname{diag}\big(f'(z_\ell)\big)\, W_\ell.$$

Los sesgos se omiten porque no intervienen en los jacobianos respecto de las activaciones. La anchura constante simplifica la notación; todos los resultados se extienden a anchuras variables.

Proposición 1.2 (Gradiente en profundidad). *Con la notación de la definición 1.1, escribiendo los gradientes como vectores columna,*

$$\nabla_{h_\ell}\mathcal{L} = J_{\ell+1}^\top J_{\ell+2}^\top \cdots J_L^\top \, \nabla_{h_L}\mathcal{L}, \qquad \frac{\partial \mathcal{L}}{\partial W_\ell} = \Big(\operatorname{diag}\big(f'(z_\ell)\big)\nabla_{h_\ell}\mathcal{L}\Big)\, h_{\ell-1}^\top.$$

Demostración. La regla de la cadena del módulo 4 para la composición $h_\ell \mapsto h_{\ell+1} \mapsto \cdots \mapsto h_L \mapsto \mathcal{L}$ da $\partial\mathcal{L}/\partial h_\ell = (\partial\mathcal{L}/\partial h_L)\, J_L J_{L-1}\cdots J_{\ell+1}$ como vector fila; trasponiendo se obtiene la primera igualdad. Para la segunda, $(z_\ell)_i = \sum_j (W_\ell)_{ij}(h_{\ell-1})_j$, luego $\partial\mathcal{L}/\partial (W_\ell)_{ij} = (\partial\mathcal{L}/\partial (z_\ell)_i)\,(h_{\ell-1})_j$, y $\partial\mathcal{L}/\partial (z_\ell)_i = f'((z_\ell)_i)\,\partial\mathcal{L}/\partial (h_\ell)_i$ porque f actúa componente a componente. ∎

El gradiente de los pesos de la capa ℓ contiene, por tanto, el producto de los $L-\ell$ jacobianos que tiene encima. Las primeras capas atraviesan casi todos.

Ejemplo 1.3 (Caso escalar). Si $d = 1$ y cada jacobiano vale una constante $a > 0$, el gradiente que llega a h_0 es a^L veces el de h_L. La cuadro 1.1 muestra que pequeñas desviaciones de $a = 1$ se amplifican exponencialmente con la profundidad.

Factor por capa a	$L = 10$	$L = 24$	Efecto
0,25	$9{,}5 \cdot 10^{-7}$	$3{,}6 \cdot 10^{-15}$	desvanecimiento extremo
0,5	$9{,}8 \cdot 10^{-4}$	$6{,}0 \cdot 10^{-8}$	las primeras capas no aprenden
1	1	1	caso ideal
1,1	2,59	9,85	crecimiento moderado
1,5	57,7	$1{,}7 \cdot 10^{4}$	explosión

Cuadro 1.1: Factor a^L por el que se multiplica el gradiente al atravesar L capas escalares.

1.2 Cotas mediante valores singulares

Para matrices, el papel del factor a lo desempeñan los valores singulares. Para $A \in \mathbb{R}^{d\times d}$ se escribe $\|A\|_2 = \max_{\|x\|=1}\|Ax\|$ (el mayor valor singular, módulo 3) y $\sigma_{\min}(A) = \min_{\|x\|=1}\|Ax\|$ (el menor). Ambos coinciden para A y $A^\top$.

Lema 1.4. *Para $A, B \in \mathbb{R}^{d\times d}$ se cumple $\sigma_{\min}(A)\,\sigma_{\min}(B) \le \sigma_{\min}(AB) \le \|AB\|_2 \le \|A\|_2\|B\|_2$.*

Demostración. Sea x con $\|x\| = 1$. Por definición, $\|Ay\| \le \|A\|_2\|y\|$ y $\|Ay\| \ge \sigma_{\min}(A)\|y\|$ para todo y. Con $y = Bx$: $\|ABx\| \le \|A\|_2\|Bx\| \le \|A\|_2\|B\|_2$ y $\|ABx\| \ge \sigma_{\min}(A)\|Bx\| \ge \sigma_{\min}(A)\sigma_{\min}(B)$. Tomando el máximo y el mínimo sobre x se obtienen las desigualdades extremas; la central es inmediata. ∎

Teorema 1.5 (Cotas del gradiente en profundidad). *Con la notación de la definición 1.1,*

$$\prod_{\ell=1}^{L}\sigma_{\min}(J_\ell) \;\le\; \frac{\|\nabla_{h_0}\mathcal{L}\|}{\|\nabla_{h_L}\mathcal{L}\|} \;\le\; \prod_{\ell=1}^{L}\|J_\ell\|_2 \;\le\; \prod_{\ell=1}^{L}\gamma\,\|W_\ell\|_2, \qquad \gamma = \sup_{t\in\mathbb{R}}|f'(t)|.$$

Demostración. Por la proposición 1.2, $\nabla_{h_0}\mathcal{L} = J_1^\top \cdots J_L^\top \nabla_{h_L}\mathcal{L}$. Aplicando el lema 1.4 $L-1$ veces y usando que $\sigma_{\min}$ y $\|\cdot\|_2$ no cambian al trasponer se obtienen las dos primeras desigualdades. La última se sigue de $\|J_\ell\|_2 \le \|\mathrm{diag}(f'(z_\ell))\|_2\|W_\ell\|_2$ y de que la norma espectral de una matriz diagonal es el mayor valor absoluto de su diagonal, que es a lo sumo γ. ∎

Corolario 1.6 (Condición suficiente de desvanecimiento). *Si existe $\lambda < 1$ tal que $\gamma\|W_\ell\|_2 \le \lambda$ para todo ℓ, entonces $\|\nabla_{h_0}\mathcal{L}\| \le \lambda^L\|\nabla_{h_L}\mathcal{L}\|$, que tiende a 0 exponencialmente con L. En consecuencia, que el gradiente explote exige que $\|W_\ell\|_2 > 1/\gamma$ para alguna capa.*

Demostración. La primera afirmación es la cota superior del teorema 1.5. La segunda es su contrarrecíproca: si $\|W_\ell\|_2 \le 1/\gamma$ en todas las capas, el cociente de normas es a lo sumo 1 y el gradiente no puede crecer. ∎

En una red recurrente la misma matriz W se aplica una vez por paso de tiempo, de modo que el corolario se lee con L igual a la longitud de la secuencia. Es el resultado de [18, §2.1]: con $\gamma = 1$ para tanh y $\gamma = 1/4$ para la sigmoide logística, basta $\|W\|_2 < 1/\gamma$ para que el gradiente se desvanezca a lo largo del tiempo. Goodfellow *et al.* [9, §10.7] y Zhang *et al.* [26, §9.7.2] presentan el caso lineal, en el que el gradiente contiene las potencias W^t.

Ejemplo 1.7 (Potencias de una matriz simétrica). Sea $M = \left(\begin{smallmatrix} 0{,}85 & 0{,}35 \\ 0{,}35 & 0{,}85 \end{smallmatrix}\right)$, con autovalores 1,2 y 0,5 y autovectores $(1,1)/\sqrt{2}$ y $(1,-1)/\sqrt{2}$. Para $x = (1,0)$,

$$M^k x = \tfrac{1}{2}\big(1{,}2^k + 0{,}5^k,\ 1{,}2^k - 0{,}5^k\big).$$

Con $k = 10$: $1{,}2^{10} = 6{,}192$ y $0{,}5^{10} = 9{,}8 \cdot 10^{-4}$, luego $M^{10}x = (3{,}096;\ 3{,}095)$. La componente en la dirección del autovalor 0,5 ha desaparecido y la norma crece como $1{,}2^k$: una misma matriz hace explotar unas direcciones y desvanecer otras.

Observación 1.8. Los productos de muchos factores grandes crean en la superficie de pérdida regiones casi verticales, llamadas *acantilados* [9, §8.2.4]. Un paso de gradiente en una de ellas puede expulsar los parámetros lejos de la región que se estaba explorando. El recorte del gradiente del tema 4 es el remedio estándar.

1.3 Saturación de las activaciones

El factor $\operatorname{diag}(f'(z_\ell))$ del jacobiano depende de las preactivaciones. Para las activaciones del módulo 9:

- $\tanh'(t) = 1 - \tanh^2(t) \in (0, 1]$, con máximo en $t = 0$. Vale 0,420 en $t = 1$, 0,0707 en $t = 2$ y 0,0099 en $t = 3$.
- La sigmoide $s(t) = 1/(1 + e^{-t})$ cumple $s'(t) = s(t)(1 - s(t)) \le 1/4$.
- $\mathrm{ReLU}'(t) = 1$ si $t > 0$ y 0 si $t < 0$.

Una unidad con $|z|$ grande está *saturada*: su derivada es casi nula y anula la fila correspondiente del jacobiano. Si una unidad ReLU tiene preactivación negativa para todas las entradas, su gradiente es exactamente cero y deja de aprender. Por el teorema 1.5, basta que una sola capa tenga $\sigma_{\min}(J_\ell)$ pequeño para que el producto completo pueda serlo.

1.4 Propagación de la varianza e inicialización

La escala de los pesos se elige para que la varianza de la señal no crezca ni decrezca de una capa a la siguiente. El resultado clave es el siguiente [26, §5.4.2].

Teorema 1.9 (Propagación de la varianza). *Sea $y = \sum_{i=1}^{n} w_i x_i$, donde las $2n$ variables $w_1, \dots, w_n, x_1, \dots, x_n$ son independientes, $\mathbb{E}[w_i] = 0$, $\operatorname{Var}(w_i) = \sigma^2$ y $\mathbb{E}[x_i^2] = q$ para todo i. Entonces $\mathbb{E}[y] = 0$ y*

$$\operatorname{Var}(y) = n\,\sigma^2\,q.$$

Si además $\mathbb{E}[x_i] = 0$, entonces $q = \operatorname{Var}(x_i)$.

Demostración. Por independencia, $\mathbb{E}[w_i x_i] = \mathbb{E}[w_i]\mathbb{E}[x_i] = 0$, luego $\mathbb{E}[y] = 0$. Para $i \neq j$, $\mathbb{E}[(w_i x_i)(w_j x_j)] = \mathbb{E}[w_i]\mathbb{E}[w_j]\mathbb{E}[x_i x_j] = 0$: los sumandos están incorrelados y sus varianzas se

suman. Cada una vale $\mathrm{Var}(w_i x_i) = \mathbb{E}[w_i^2 x_i^2] = \mathbb{E}[w_i^2]\mathbb{E}[x_i^2] = \sigma^2 q$. Sumando los n términos se obtiene el enunciado. ■

Corolario 1.10 (Inicialización de Xavier). *En una red lineal ($f(t) = t$) con entradas centradas,* $\mathrm{Var}(z_\ell) = (n\sigma^2)^\ell \mathrm{Var}(h_0)$. *La varianza se conserva en todas las capas si y solo si* $\sigma^2 = 1/n$. *La pasada hacia atrás multiplica por las matrices traspuestas, cuyo número de entradas por fila es el de salidas, y la varianza del gradiente se conserva con* $\sigma^2 = 1/n_{sal}$; *Glorot y Bengio [7] proponen el compromiso* $\sigma^2 = 2/(n_{ent} + n_{sal})$.

Demostración. Aplicando el teorema 1.9 capa a capa, cada una multiplica la varianza por $n\sigma^2$; tras ℓ capas el factor es $(n\sigma^2)^\ell$, que vale 1 para todo ℓ si y solo si $n\sigma^2 = 1$. ■

Proposición 1.11 (Inicialización de He). *Sea z una variable aleatoria con densidad simétrica respecto de 0 y varianza v. Entonces* $\mathbb{E}[\mathrm{ReLU}(z)^2] = v/2$. *En consecuencia, en una red ReLU la varianza de las preactivaciones se conserva con* $\sigma^2 = 2/n$ *[11].*

Demostración. Por simetría, $\int_0^\infty t^2\rho(t)\,dt = \int_{-\infty}^0 t^2\rho(t)\,dt$, y su suma es $\mathbb{E}[z^2] = v$. Como $\mathrm{ReLU}(z)^2$ vale z^2 si $z > 0$ y 0 en otro caso, $\mathbb{E}[\mathrm{ReLU}(z)^2] = \int_0^\infty t^2\rho(t)\,dt = v/2$. Por el teorema 1.9 con $q = v/2$, la preactivación siguiente tiene varianza $n\sigma^2 v/2$, igual a v si y solo si $\sigma^2 = 2/n$. ■

Ejemplo 1.12 (Inicialización por defecto de una capa lineal). Una capa lineal de PyTorch inicializa sus pesos con distribución uniforme en $[-1/\sqrt{n},\, 1/\sqrt{n}]$. La varianza de la uniforme en $[-a, a]$ es $a^2/3$, luego $\sigma^2 = 1/(3n)$ y $n\sigma^2 = 1/3$. En una red lineal de 24 capas la desviación típica de la señal se multiplica por $(1/\sqrt{3})^{24} = 3^{-12} \approx 1{,}9 \cdot 10^{-6}$, y el gradiente sufre un factor del mismo orden. En la práctica del módulo, una red de 24 capas tanh con esta inicialización da un cociente de $1{,}4 \cdot 10^{-5}$ entre la norma del gradiente de la primera capa y la de la última.

Observación 1.13 (Conexión con los LLM). Las redes recurrentes del módulo 12 aplican el mismo jacobiano una vez por token, y el corolario 1.6 explica su dificultad para aprender dependencias largas. El Transformer evita la recurrencia, pero apila decenas de bloques; lo que lo hace entrenable es la conexión residual del tema 2, no una inicialización perfecta.

Lecturas recomendadas para este tema

- Goodfellow, Bengio y Courville [9], §8.2.4 «Cliffs and Exploding Gradients», §8.2.5 «Long-Term Dependencies», §8.4 «Parameter Initialization Strategies» y §10.7 «The Challenge of Long-Term Dependencies».
- Zhang *et al.* [26], §5.4 «Numerical Stability and Initialization» y §9.7 «Backpropagation Through Time».
- Bishop y Bishop [2], §7.2.5 «Parameter initialization».
- Pascanu, Mikolov y Bengio [18], §2.1 «The mechanics».
- Notas de CS231n [3], «Neural Networks Part 2», apartado «Weight Initialization».

Ejercicios del tema 1

Ejercicio 1.1 ⋆ Explique por qué, en una red de 24 capas sin conexiones residuales, la primera capa puede quedar prácticamente «congelada» aunque la última aprenda con normalidad. Use la proposición 1.2 y el ejemplo 1.12.

Ejercicio 1.2 ⋆⋆ Calcule el factor por el que se multiplica el gradiente tras 24 capas escalares si cada una contribuye con (a) 0,5; (b) 1,1. (c) ¿Cuántas capas con factor 0,5 hacen falta para que el factor caiga por debajo de $2^{-24} \approx 6 \cdot 10^{-8}$, el menor número positivo representable en fp16 (tema 5)?

Ejercicio 1.3 ⋆⋆ (a) Calcule $\tanh'(t)$ para $t = 0, 1, 2, 3$. (b) En una red escalar de 5 capas tanh con pesos $w_\ell = 1$, todas las preactivaciones valen 2. Calcule el factor del gradiente entre la salida y la entrada. (c) ¿Qué factor daría la sigmoide en el mejor caso posible?

Ejercicio 1.4 ⋆⋆ Una capa tiene $n = 64$ entradas. (a) Calcule la desviación típica σ de los pesos que conserva la varianza en una red lineal y la que la conserva en una red ReLU. (b) Calcule el factor $n\sigma^2$ de la inicialización por defecto del ejemplo 1.12 y la varianza de la señal tras 10 capas lineales, partiendo de varianza 1.

Ejercicio 1.5 ⋆⋆⋆ (a) Demuestre que, para matrices cuadradas, todos los valores singulares de AB están comprendidos entre $\sigma_{\min}(A)\sigma_{\min}(B)$ y $\|A\|_2\|B\|_2$. (b) Deduzca que si los jacobianos de una red cumplen $s_{\min} \le \sigma_{\min}(J_\ell)$ y $\|J_\ell\|_2 \le s_{\max}$, el cociente del teorema 1.5 está entre $s_{\min}^L$ y $s_{\max}^L$, y calcule ambos extremos para $s_{\min} = 0{,}9$, $s_{\max} = 1{,}1$ y $L = 24$. (c) ¿Qué indica el resultado sobre las condiciones que deben cumplir los jacobianos para que el gradiente esté bien condicionado?

Fuente: adaptado de Zhang *et al.* [26], §5.4, ejercicio 3 (cotas de los autovalores de un producto de matrices).

Ejercicio 1.6 ⋆⋆⋆ Sea $M \in \mathbb{R}^{d\times d}$ simétrica con autovalores $\lambda_1 > |\lambda_2| \ge \cdots \ge |\lambda_d|$ y autovectores ortonormales $v_1, \ldots, v_d$. (a) Demuestre que M^k tiene autovalores λ_i^k con los mismos autovectores. (b) Demuestre que, si x tiene componente no nula sobre v_1, la dirección de $M^k x$ converge a la de $\pm v_1$. (c) Compruebe (b) con la matriz del ejemplo 1.7 para $k = 1$ y $k = 10$. (d) Explique qué implica para los gradientes de una red recurrente.

Fuente: adaptado de Zhang *et al.* [26], §9.7, ejercicio 1.

Tema 2

Conexiones residuales

Una conexión residual suma a la salida de un bloque su propia entrada. Con este cambio mínimo, el jacobiano de cada bloque pasa a ser la identidad más una corrección, y el producto de jacobianos del tema 1 contiene siempre un término que no se atenúa con la profundidad. Este tema define las redes residuales, demuestra sus propiedades de propagación hacia delante y hacia atrás, y describe el flujo residual de un Transformer. Se sigue el artículo original de He *et al.* [10], su continuación sobre aplicaciones identidad [12], [26, §8.6] y [2, §9.5].

2.1 Clases de funciones anidadas

Zhang *et al.* [26, §8.6.1] motivan las redes residuales con un argumento sobre clases de funciones. Sea $\mathcal{F}_L$ el conjunto de funciones que puede representar una arquitectura de L bloques. Al añadir un bloque se esperaría $\mathcal{F}_L \subseteq \mathcal{F}_{L+1}$: una red más profunda debería poder imitar a la menos profunda. En una red plana esto exige que el bloque nuevo aprenda exactamente la identidad, lo que no es fácil con capas no lineales; empíricamente, las redes planas muy profundas obtienen *mayor* error de entrenamiento que otras menos profundas [10].

Definición 2.1 (Bloque y red residual). Un *bloque residual* es una aplicación $\mathbb{R}^d \to \mathbb{R}^d$ de la forma

$$h \longmapsto h + F(h;\theta),$$

donde F, la *rama residual*, es una red diferenciable con parámetros θ. Una *red residual* de L bloques calcula $h_\ell = h_{\ell-1} + F_\ell(h_{\ell-1})$ para $\ell = 1, \dots, L$.

Proposición 2.2 (Anidamiento). *Si cada rama F_ℓ termina en una capa lineal $u \mapsto Vu$, la red residual de $L+1$ bloques puede representar cualquier función que represente la de L bloques.*

Demostración. Basta tomar $V = 0$ en la última capa de F_{L+1}. Entonces $F_{L+1} \equiv 0$ y el bloque añadido es la identidad, de modo que $h_{L+1} = h_L$. ∎

La proposición explica también la idea de *residuo*: el bloque no tiene que aprender la transformación deseada $H(h)$, sino la diferencia $F(h) = H(h) - h$, que es pequeña cuando H se parece a la identidad.

2.2 Propagación en una red residual

Proposición 2.3 (Propagación hacia delante). *En una red residual, para $0 \le \ell < L$,*

$$h_L = h_\ell + \sum_{k=\ell+1}^{L} F_k(h_{k-1}).$$

Demostración. Por inducción sobre $L-\ell$. Si $L=\ell+1$ es la definición 2.1. Si es cierto para $L-1$, entonces $h_L = h_{L-1} + F_L(h_{L-1}) = h_\ell + \sum_{k=\ell+1}^{L-1} F_k(h_{k-1}) + F_L(h_{L-1})$. ■

La salida es la entrada de cualquier bloque más la suma de las aportaciones de los bloques posteriores. Esta lectura aditiva, destacada en [12, §2], contrasta con la composición $f_L \circ \cdots \circ f_1$ de una red plana.

Teorema 2.4 (Jacobiano de una red residual). *Sea $J_\ell = \partial F_\ell / \partial h_{\ell-1}$, evaluado en $h_{\ell-1}$. Entonces*

$$\frac{\partial h_L}{\partial h_0} = (I+J_L)(I+J_{L-1})\cdots(I+J_1) = I + \sum_{\ell=1}^{L} J_\ell + \sum_{1\le \ell<k\le L} J_k J_\ell + \cdots + J_L \cdots J_1,$$

donde el desarrollo tiene 2^L términos, uno por cada subconjunto de bloques, con los factores en orden decreciente de índice. En particular, contiene el término I.

Demostración. El jacobiano de $h \mapsto h + F_\ell(h)$ es $I + J_\ell$, porque la derivada de la identidad es I. La regla de la cadena da el producto. Al desarrollarlo, cada término se obtiene eligiendo en cada factor $(I+J_\ell)$ o bien I o bien J_ℓ; hay 2^L elecciones, y la que elige I en los L factores produce I. Como el producto de matrices no es conmutativo, en cada término se conserva el orden original de los factores. ■

Corolario 2.5. $\nabla_{h_0}\mathcal{L} = \nabla_{h_L}\mathcal{L} + \left(\sum_\ell J_\ell + \sum_{\ell<k} J_k J_\ell + \cdots\right)^\top \nabla_{h_L}\mathcal{L}$. *El gradiente de la salida llega a la entrada sin atenuar, sumado a correcciones.*

Cada término del desarrollo corresponde a un camino por la red que atraviesa un subconjunto de ramas y salta las demás. Una red plana solo dispone del camino $J_L \cdots J_1$, que es el último término.

Ejemplo 2.6 (Dos bloques escalares). Con $J_1 = -0{,}3$ y $J_2 = 0{,}2$, la red residual da $(1-0{,}3)(1+0{,}2) = 0{,}84$, que es $1 + (-0{,}3+0{,}2) + (0{,}2)(-0{,}3) = 1 - 0{,}1 - 0{,}06$. La red plana con los mismos jacobianos daría solo $J_2 J_1 = -0{,}06$.

Proposición 2.7 (Cota inferior). *Si $\|J_\ell\|_2 \le a < 1$ para todo ℓ, entonces*

$$\sigma_{\min}\left(\frac{\partial h_L}{\partial h_0}\right) \ge (1-a)^L > 0.$$

En particular, ningún vector de gradiente no nulo en la salida se anula en la entrada.

Demostración. Para $\|x\| = 1$, la desigualdad triangular inversa da $\|(I+J_\ell)x\| \ge \|x\| - \|J_\ell x\| \ge 1-a$, luego $\sigma_{\min}(I+J_\ell) \ge 1-a$. Por el lema 1.4, el menor valor singular del producto es al menos $(1-a)^L$. La segunda afirmación se sigue del corolario 2.5 y de que $\sigma_{\min}$ no cambia al trasponer. ■

En una red plana, en cambio, basta un J_ℓ con núcleo no trivial (una capa ReLU con unidades apagadas, por ejemplo) para que algún gradiente se anule. La cota de la proposición 2.7 no impide el desvanecimiento si a es grande, pero al inicio del entrenamiento las ramas tienen jacobianos pequeños y $(1 \pm a)^L$ queda cerca de 1: con $a = 0{,}01$ y $L = 24$, entre $0{,}99^{24} = 0{,}786$ y $1{,}01^{24} = 1{,}27$.

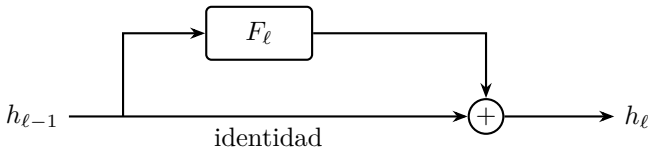


Figura 2.1: Bloque residual. El camino inferior aporta el término I del teorema 2.4; la rama superior aporta J_ℓ.

2.3 El flujo residual de un Transformer

En un Transformer, el vector $h_\ell \in \mathbb{R}^d$ asociado a cada token recorre todos los bloques. Cada subbloque de atención o de perceptrón multicapa lee de él y le suma su aportación; por eso se le llama *flujo residual*. La proposición 2.3 dice que el flujo de la última capa es la suma del embedding inicial y de todas las aportaciones. En el módulo 23 se usará esta descomposición para interpretar qué escribe cada bloque.

Proposición 2.8 (Crecimiento del flujo residual). *Supóngase que, al inicio del entrenamiento, las aportaciones $F_k(h_{k-1})$ de una componente del flujo son variables aleatorias de media nula, incorreladas entre sí y con h_0, y con varianza v cada una. Entonces* $\mathrm{Var}\big((h_L)_i\big) = \mathrm{Var}\big((h_0)_i\big) + Lv$*: la desviación típica del flujo crece como* $\sqrt{L}$.

Demostración. Por la proposición 2.3, $(h_L)_i = (h_0)_i + \sum_{k=1}^{L} F_k(h_{k-1})_i$. La varianza de una suma de variables incorreladas es la suma de sus varianzas. ∎

Este crecimiento motiva dos prácticas de los modelos de lenguaje. GPT-2 [19, §2.3] multiplica en la inicialización los pesos de las capas que escriben en el flujo por $1/\sqrt{N}$, con N el número de capas residuales, lo que reduce cada varianza v en un factor N. Y los Transformer pre-norm añaden una normalización final antes de la capa de salida (tema 3).

Observación 2.9 (Conexión con los LLM). Todos los Transformer abiertos que se estudian en el curso (GPT-2, LLaMA, Grok-1) usan conexiones residuales en torno a cada subbloque. Las ResNet se introdujeron en visión para entrenar redes de 152 capas [10]; en los modelos de lenguaje, la misma idea permite apilar decenas de bloques.

Lecturas recomendadas para este tema

- He, Zhang, Ren y Sun [10], §3 «Deep Residual Learning», y [12], §2 (propagación con atajos identidad).
- Zhang *et al.* [26], §8.6.1 «Function Classes» y §8.6.2 «Residual Blocks».
- Bishop y Bishop [2], §9.5 «Residual Connections».
- Radford *et al.* [19], §2.3 «Model» (inicialización escalada de las capas residuales).

Ejercicios del tema 2

Ejercicio 2.1 ⋆ La proposición 2.2 garantiza que añadir bloques residuales no reduce la clase de funciones representables. ¿Por qué no basta entonces con aumentar sin límite la profundidad? Dé dos razones, una estadística y otra computacional.

Fuente: adaptado de Zhang *et al.* [26], §8.6, ejercicio 5.

Ejercicio 2.2 ⋆⋆ En una red residual escalar de 24 bloques, calcule el factor $\prod_{\ell}(1+J_\ell)$ si (a) $J_\ell = 0{,}01$ para todo ℓ; (b) $J_\ell = -0{,}01$ para todo ℓ. (c) Compárelos con la red plana de 24 capas con factor 0,5 y explique por qué la red residual con $J_\ell = -0{,}5$ sería igual de mala.

Ejercicio 2.3 ⋆⋆ Sean $J_1 = \left(\begin{smallmatrix} 0{,}2 & 0 \\ 0{,}1 & -0{,}1 \end{smallmatrix}\right)$ y $J_2 = \left(\begin{smallmatrix} -0{,}1 & 0{,}3 \\ 0 & 0{,}2 \end{smallmatrix}\right)$. Calcule el jacobiano $(I+J_2)(I+J_1)$ de una red residual de dos bloques, compruebe que es igual a $I + J_1 + J_2 + J_2J_1$ y compárelo con el jacobiano J_2J_1 de la red plana.

Ejercicio 2.4 ⋆⋆ Un Transformer tiene $N = 48$ capas residuales. Al inicio, el flujo tiene varianza 1 por componente y cada capa aporta una variable incorrelada de varianza $v = 0{,}5$. (a) Calcule la varianza y la desviación típica del flujo a la salida. (b) Repita el cálculo si los pesos de salida de cada capa se multiplican por $1/\sqrt{N}$, como en GPT-2.

Ejercicio 2.5 ⋆⋆⋆ (a) Demuestre que el desarrollo del teorema 2.4 contiene el término I. (b) En el caso escalar con $|J_\ell| \le a < 1$, demuestre que $\prod_{\ell}(1+J_\ell) \ge (1-a)^{L} > 0$. (c) Muestre con un ejemplo de dos capas que en una red plana el factor $\prod_{\ell} J_\ell$ puede anularse aunque solo una capa tenga jacobiano nulo.

Tema 3

Normalización: LayerNorm, RMSNorm y pre-norm

Normalizar una capa consiste en reescalar cada vector de activaciones con estadísticos calculados sobre él mismo, de modo que su escala no derive de una capa a otra. Este tema define la normalización por lotes, la normalización por capa (LayerNorm) y su simplificación RMSNorm, demuestra sus propiedades de invariancia, calcula el jacobiano de RMSNorm y analiza dónde debe colocarse la normalización respecto de la conexión residual. Se sigue [2, §7.4], [26, §8.5] y [9, §8.7.1], junto con los artículos de Ioffe y Szegedy [15], Ba, Kiros y Hinton [1], Zhang y Sennrich [27] y Xiong *et al.* [25].

3.1 Normalización por lotes y por capa

Definición 3.1 (Normalización por lotes). Sea un minibatch de B vectores $x^{(1)}, \dots, x^{(B)} \in \mathbb{R}^d$. La *normalización por lotes* (BatchNorm) normaliza cada *componente* i con la media y la varianza de esa componente sobre el lote:

$$\hat{x}_i^{(b)} = \frac{x_i^{(b)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \qquad \mu_i = \frac{1}{B}\sum_{b=1}^{B} x_i^{(b)}, \quad \sigma_i^2 = \frac{1}{B}\sum_{b=1}^{B}\big(x_i^{(b)} - \mu_i\big)^2,$$

y después aplica una ganancia y un sesgo aprendidos por componente [15].

La salida de BatchNorm para un ejemplo depende de los demás ejemplos del lote; en inferencia se sustituyen μ_i y σ_i^2 por medias móviles estimadas durante el entrenamiento [26, §8.5.2]. Para secuencias de longitud variable y para la generación token a token, esta dependencia es un inconveniente. La normalización por capa la elimina normalizando sobre las componentes de cada vector.

Definición 3.2 (LayerNorm). Sea $x \in \mathbb{R}^d$, y sean $\gamma, \beta \in \mathbb{R}^d$ parámetros y $\epsilon \geq 0$. Con

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \qquad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2,$$

la *normalización por capa* es

$$\mathrm{LN}(x)_i = \gamma_i \, \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i, \qquad i = 1, \dots, d \ [1].$$

Definición 3.3 (RMSNorm). Sea $x \in \mathbb{R}^d$, $g \in \mathbb{R}^d$ y $\epsilon \geq 0$. Con $\mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2 + \epsilon}$, se define

$$\mathrm{RMSNorm}(x)_i = g_i \, \frac{x_i}{\mathrm{RMS}(x)}, \qquad i = 1, \dots, d \ [27].$$

RMSNorm suprime el centrado y el sesgo β: solo fija la escala. En un Transformer ambas se aplican por separado al vector de cada token, por lo que funcionan igual en entrenamiento y en inferencia y con lotes de cualquier tamaño.

Ejemplo 3.4. Sea $x = (2, 4, 6)$ con $\gamma = g = \mathbf{1}$, $\beta = 0$ y $\epsilon = 0$. Para LayerNorm, $\mu = 4$, $\sigma^2 = (4 + 0 + 4)/3 = 8/3$ y $\sigma = 1{,}633$, luego $\mathrm{LN}(x) = (-1{,}225;\ 0;\ 1{,}225)$. Para RMSNorm, $\mathrm{RMS}(x) = \sqrt{(4 + 16 + 36)/3} = \sqrt{56/3} = 4{,}320$, luego $\mathrm{RMSNorm}(x) = (0{,}463;\ 0{,}926;\ 1{,}389)$.

Proposición 3.5 (Estadísticos de la salida). *Sea $\epsilon = 0$, $\gamma = g = \mathbf{1}$ y $\beta = 0$.*

(i) Si x no es constante, $y = \mathrm{LN}(x)$ cumple $\frac{1}{d}\sum_i y_i = 0$ y $\frac{1}{d}\sum_i y_i^2 = 1$.

(ii) Si $x \neq 0$, $y = \mathrm{RMSNorm}(x)$ cumple $\frac{1}{d}\sum_i y_i^2 = 1$, es decir, $\|y\| = \sqrt{d}$.

Demostración. (i) $\sum_i (x_i - \mu) = \sum_i x_i - d\mu = 0$, y $\frac{1}{d}\sum_i (x_i - \mu)^2/\sigma^2 = \sigma^2/\sigma^2 = 1$. (ii) $\frac{1}{d}\sum_i x_i^2 / \mathrm{RMS}(x)^2 = \mathrm{RMS}(x)^2/\mathrm{RMS}(x)^2 = 1$. ∎

3.2 Invariancias

Proposición 3.6 (Invariancias). *Sea $\epsilon = 0$, $\alpha > 0$ y $c \in \mathbb{R}$.*

(i) $\mathrm{LN}(\alpha x + c\mathbf{1}) = \mathrm{LN}(x)$ *para todo x no constante.*

(ii) $\mathrm{RMSNorm}(\alpha x) = \mathrm{RMSNorm}(x)$ *para todo $x \neq 0$, pero en general* $\mathrm{RMSNorm}(x + c\mathbf{1}) \neq \mathrm{RMSNorm}(x)$.

(iii) Si la entrada de la normalización es $x = Wu$ con $W \in \mathbb{R}^{d\times n}$, la salida no cambia al sustituir W por αW, para LayerNorm y para RMSNorm.

Demostración. (i) La media de $\alpha x + c\mathbf{1}$ es $\alpha\mu + c$ y su varianza $\alpha^2\sigma^2$; el cociente $(\alpha x_i + c - \alpha\mu - c)/(\alpha\sigma)$ es $(x_i - \mu)/\sigma$. (ii) $\mathrm{RMS}(\alpha x) = \alpha\,\mathrm{RMS}(x)$ y el cociente no cambia. Para el contraejemplo, con $x = (1, 3)$ se tiene $\mathrm{RMS}(x) = \sqrt{5}$ y $\mathrm{RMSNorm}(x) = (0{,}447;\ 1{,}342)$, mientras que $x + \mathbf{1} = (2, 4)$ tiene $\mathrm{RMS} = \sqrt{10}$ y $\mathrm{RMSNorm}(x + \mathbf{1}) = (0{,}632;\ 1{,}265)$. (iii) Es (i) o (ii) aplicado a $\alpha W u = \alpha x$. ∎

La propiedad (iii) figura en la tabla de invariancias de Ba *et al.* [1, §5.1]: la salida no depende de la escala de los pesos de la capa anterior. Zhang y Sennrich [27] conservan esta invariancia de escala, que consideran la responsable del buen comportamiento de LayerNorm, y renuncian a la de traslación, con lo que se ahorra el cálculo de la media.

Teorema 3.7 (Jacobiano de RMSNorm). *Sea $\epsilon = 0$, $x \neq 0$ y $r = \mathrm{RMS}(x)$. El jacobiano de $y = \mathrm{RMSNorm}(x)$ es*

$$J = \frac{\partial y}{\partial x} = \frac{1}{r}\,\mathrm{diag}(g)\Big(I - \frac{x x^\top}{d\, r^2}\Big) = \frac{1}{r}\,\mathrm{diag}(g)\Big(I - \frac{x x^\top}{\|x\|^2}\Big).$$

En consecuencia, $Jx = 0$ y, para todo $v \in \mathbb{R}^d$, $x^\top(J^\top v) = 0$: el gradiente que atraviesa RMSNorm hacia atrás es ortogonal a x, y su norma es a lo sumo $\max_i |g_i|\,\|v\|/r$.

Demostración. Como $r^2 = \frac{1}{d}\sum_k x_k^2$, se tiene $\partial r/\partial x_j = x_j/(d\,r)$. Derivando $y_i = g_i x_i / r$,

$$\frac{\partial y_i}{\partial x_j} = g_i\Big(\frac{\delta_{ij}}{r} - \frac{x_i}{r^2}\frac{x_j}{d\,r}\Big) = \frac{g_i}{r}\Big(\delta_{ij} - \frac{x_i x_j}{d\, r^2}\Big),$$

que es la expresión matricial del enunciado, con $d\,r^2 = \|x\|^2$. La matriz $P = I - xx^\top/\|x\|^2$ es la proyección ortogonal sobre el complemento de x, luego $Px = 0$ y $Jx = 0$. Además $J^\top v = \frac{1}{r} P\,\mathrm{diag}(g)v$ y $x^\top P = (Px)^\top = 0$. Por último, $\|P\|_2 = 1$ y $\|\mathrm{diag}(g)\|_2 = \max_i |g_i|$. ■

La identidad $Jx = 0$ es la versión infinitesimal de la invariancia de escala: al derivar $\mathrm{RMSNorm}((1+t)x) = \mathrm{RMSNorm}(x)$ en $t = 0$ se obtiene $Jx = 0$. La cota $\|v\|/r$ muestra el efecto estabilizador: si una capa anterior infla sus activaciones, r crece y el gradiente que se le devuelve se reduce en la misma proporción.

Observación 3.8. Un cálculo análogo, con $\hat{y} = (x - \mu\mathbf{1})/\sigma$, da para LayerNorm con $\gamma = \mathbf{1}$, $\beta = 0$ y $\epsilon = 0$

$$\frac{\partial \hat{y}}{\partial x} = \frac{1}{\sigma}\Big(I - \frac{\mathbf{1}\mathbf{1}^\top}{d} - \frac{\hat{y}\hat{y}^\top}{d}\Big),$$

que anula tanto $\mathbf{1}$ como $x - \mu\mathbf{1}$, de acuerdo con la invariancia (i) de la proposición 3.6.

3.3 Post-norm y pre-norm

Definición 3.9 (Posición de la normalización). Sea Norm una normalización (LayerNorm o RMSNorm) y F una rama residual. Un bloque es

$$\textit{post-norm} \text{ si } h_\ell = \mathrm{Norm}\big(h_{\ell-1} + F(h_{\ell-1})\big), \qquad \textit{pre-norm} \text{ si } h_\ell = h_{\ell-1} + F\big(\mathrm{Norm}(h_{\ell-1})\big).$$

El Transformer original [23] es post-norm. GPT-2 [19, §2.3] trasladó la normalización a la entrada de cada subbloque y añadió una normalización final tras el último bloque; LLaMA [22] usa pre-norm con RMSNorm, y la mayoría de los modelos abiertos posteriores la han adoptado.

Proposición 3.10 (Jacobianos de ambas posiciones). *Sea N_ℓ el jacobiano de la normalización del bloque ℓ y J_ℓ el de la rama F_ℓ, cada uno evaluado en su argumento. Entonces*

$$\textit{post-norm:}\ \frac{\partial h_L}{\partial h_0} = \prod_{\ell=L}^{1} N_\ell\,(I + J_\ell), \qquad \textit{pre-norm:}\ \frac{\partial h_L}{\partial h_0} = \prod_{\ell=L}^{1} \big(I + J_\ell N_\ell\big),$$

donde los productos se escriben con los índices en orden decreciente. Solo el desarrollo de pre-norm contiene el término I.

Demostración. Regla de la cadena aplicada a cada definición: en post-norm el bloque es $\mathrm{Norm} \circ (\mathrm{id} + F)$; en pre-norm, $\mathrm{id} + F \circ \mathrm{Norm}$. En post-norm, todo término del desarrollo contiene los L factores N_ℓ, que no son la identidad (por el teorema 3.7, cada uno anula una dirección). En pre-norm, eligiendo I en cada factor se obtiene I, como en el teorema 2.4. ■

En pre-norm, el flujo residual no pasa por ninguna normalización y conserva el camino identidad. A cambio, su norma crece con la profundidad (proposición 2.8), y hace falta una normalización final antes de la proyección a logits. Xiong *et al.* [25] demuestran, con argumentos de campo medio, que en post-norm el gradiente esperado de los parámetros próximos a la salida es grande en la inicialización, lo que obliga a un calentamiento largo de la tasa de aprendizaje (módulo 7); en pre-norm los gradientes están acotados y el calentamiento puede acortarse o suprimirse.

Observación 3.11 (Sobre los modelos comerciales). Anthropic y Google no han hecho pública la normalización que usan Claude y Gemini. El código publicado de Grok-1 [24] incluye capas RMSNorm.

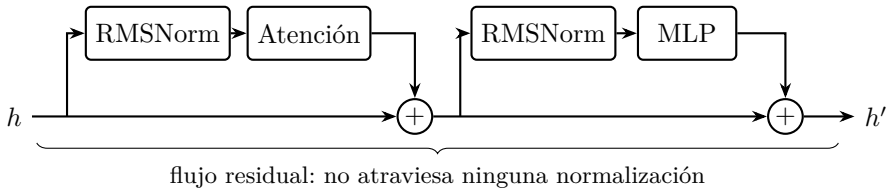


Figura 3.1: Bloque decodificador pre-norm, el que se construye en el módulo 14. Cada subbloque normaliza su entrada y suma su salida al flujo residual.

Lecturas recomendadas para este tema

- Bishop y Bishop [2], §7.4.2 «Batch normalization» y §7.4.3 «Layer normalization».
- Zhang *et al.* [26], §8.5 «Batch Normalization», en particular §8.5.2.3 «Layer Normalization».
- Goodfellow, Bengio y Courville [9], §8.7.1 «Batch Normalization».
- Ba, Kiros y Hinton [1], §3 y §5.1 «Invariance under weights and data transformations».
- Zhang y Sennrich [27]; Xiong *et al.* [25].

Ejercicios del tema 3

Ejercicio 3.1 ⋆ Clasifique como pre-norm o post-norm: (a) $h + \mathrm{Attn}(\mathrm{LN}(h))$; (b) $\mathrm{LN}(h + \mathrm{MLP}(h))$. ¿Cuál conserva la identidad en el camino residual? Justifíquelo con la proposición 3.10.

Ejercicio 3.2 ⋆ En una versión posterior de las ResNet, cada rama pasó de aplicar «convolución, normalización, activación» seguidas de la suma y una activación a aplicar «normalización, activación, convolución» antes de la suma, sin nada después. Explique por qué la segunda ordenación deja un camino identidad limpio entre bloques y qué relación guarda con pre-norm.

Fuente: adaptado de Zhang *et al.* [26], §8.6, ejercicio 4 (véase también [12]).

Ejercicio 3.3 ⋆⋆ Calcule $\mathrm{LN}(x)$ y $\mathrm{RMSNorm}(x)$ para $x = (1, 2, 3, 6)$ con $\gamma = g = \mathbf{1}$, $\beta = 0$ y $\epsilon = 0$, y compruebe la proposición 3.5.

Ejercicio 3.4 ⋆⋆ Con $x = (1, 2, 3, 6)$: (a) calcule $\mathrm{LN}(2x + 10 \cdot \mathbf{1})$ y compruebe que coincide con $\mathrm{LN}(x)$; (b) calcule $\mathrm{RMSNorm}(x + 3 \cdot \mathbf{1})$ y compárelo con $\mathrm{RMSNorm}(x)$. ¿Qué parte de la proposición 3.6 ilustra cada apartado?

Ejercicio 3.5 ⋆⋆ Calcule el jacobiano de RMSNorm en $x = (3, 4)$ con $g = \mathbf{1}$ y $\epsilon = 0$ usando el teorema 3.7, y compruebe que $Jx = 0$ y que J es simétrica.

Ejercicio 3.6 ⋆⋆⋆ (a) Demuestre que $\mathrm{RMSNorm}(\alpha x) = \mathrm{RMSNorm}(x)$ para $\alpha > 0$ y $\epsilon = 0$, y deduzca $Jx = 0$ derivando respecto de α. (b) Sea $\mathcal{L}$ una pérdida que depende de x solo a través de $y = \mathrm{RMSNorm}(x)$. Demuestre que $x^\top \nabla_x \mathcal{L} = 0$. (c) Demuestre que a lo largo del flujo del gradiente $\dot{x} = -\nabla_x \mathcal{L}$ la norma $\|x\|$ es constante, y que un paso discreto $x \leftarrow x - \eta \nabla_x \mathcal{L}$ con $\eta > 0$ la aumenta salvo que el gradiente sea nulo. (d) ¿Qué ocurre con $\epsilon > 0$?

Tema 4

Dropout y recorte del gradiente

Este tema reúne dos técnicas que actúan sobre el entrenamiento sin cambiar la arquitectura. El dropout introduce ruido multiplicativo en las activaciones para regularizar; el recorte del gradiente limita la longitud de cada paso para que un gradiente anómalo no destruya lo aprendido. Se demuestran las propiedades que justifican ambas: el dropout invertido es insesgado y su varianza se calcula en forma cerrada; el recorte por norma global conserva la dirección del gradiente y acota el paso. Se sigue [9, §7.12], [26, §5.6], [2, §9.6.1] y las notas de CS231n [3] para el dropout, con el artículo de Srivastava *et al.* [21]; y [9, §8.2.4 y §10.11] y el artículo de Pascanu, Mikolov y Bengio [18] para el recorte.

4.1 Dropout

Definición 4.1 (Dropout invertido). Sea $h \in \mathbb{R}^d$ un vector de activaciones y $p \in [0, 1)$. Durante el entrenamiento, el *dropout invertido* con probabilidad p sustituye h por

$$\tilde{h}_i = \frac{m_i \, h_i}{1 - p}, \qquad m_i \sim \text{Bernoulli}(1 - p) \text{ independientes}, \qquad i = 1, \dots, d.$$

En evaluación se usa $\tilde{h} = h$.

Cada unidad se apaga con probabilidad p y las que sobreviven se amplifican por $1/(1-p)$. La variante original [21] no amplificaba durante el entrenamiento y, en cambio, multiplicaba los pesos por $1-p$ en la evaluación; la variante invertida traslada esa corrección al entrenamiento y deja intacto el código de inferencia [3].

Proposición 4.2 (Esperanza y varianza). *Con la notación de la definición 4.1, para h fijo,*

$$\mathbb{E}[\tilde{h}_i] = h_i, \qquad \operatorname{Var}(\tilde{h}_i) = \frac{p}{1 - p} \, h_i^2.$$

Demostración. $\mathbb{E}[m_i] = 1 - p$ y, como $m_i^2 = m_i$, $\mathbb{E}[m_i^2] = 1 - p$. Por linealidad, $\mathbb{E}[\tilde{h}_i] = h_i(1 - p)/(1 - p) = h_i$. Además $\mathbb{E}[\tilde{h}_i^2] = h_i^2(1 - p)/(1 - p)^2 = h_i^2/(1 - p)$, y $\operatorname{Var}(\tilde{h}_i) = h_i^2/(1 - p) - h_i^2 = h_i^2 \, p/(1 - p)$. ∎

Corolario 4.3 (Dropout antes de una capa lineal). *Sea $y = w^\top \tilde{h}$ con $w \in \mathbb{R}^d$ fijo. Entonces $\mathbb{E}[y] = w^\top h$ y $\operatorname{Var}(y) = \frac{p}{1-p} \sum_{i=1}^{d} w_i^2 h_i^2$.*

Demostración. La esperanza se sigue de la linealidad y de la proposición 4.2. Las $\tilde{h}_i$ son independientes, luego $\operatorname{Var}(y) = \sum_i w_i^2 \operatorname{Var}(\tilde{h}_i)$. ∎

El corolario muestra que, para una capa lineal, evaluar sin máscara da exactamente la salida media sobre todas las máscaras. Goodfellow *et al.* [9, §7.12] interpretan el dropout

como el entrenamiento simultáneo de los 2^d subredes que se obtienen al eliminar unidades, con parámetros compartidos; la regla de evaluación sin máscara, llamada *regla de escalado de pesos*, es exacta para modelos sin no linealidades ocultas, como el anterior, y solo aproximada en redes profundas no lineales.

Ejemplo 4.4. Con $p = 0{,}3$, las supervivientes se multiplican por $1/0{,}7 = 1{,}429$. Para $h = (5, 5, 5, 5)$ y la máscara $m = (1, 0, 1, 1)$, $\tilde{h} = (7{,}143;\ 0;\ 7{,}143;\ 7{,}143)$. Cada componente tiene esperanza 5 y, por la proposición 4.2, varianza $\frac{0{,}3}{0{,}7} \cdot 25 = 10{,}71$.

Observación 4.5 (Conexión con los LLM). El dropout reduce el sobreajuste, que es pequeño cuando cada ejemplo se ve una sola vez. En el preentrenamiento de modelos de lenguaje con billones de tokens se usa dropout bajo o nulo; en el ajuste fino con pocos datos (módulo 20) vuelve a ser útil. En evaluación hay que desactivarlo: en PyTorch, con `model.eval()`.

4.2 Recorte del gradiente por norma global

Definición 4.6 (Recorte por norma global). Sean $g^{(1)}, \dots, g^{(K)}$ los gradientes de los K tensores de parámetros de un modelo y sea $g \in \mathbb{R}^n$ su concatenación. Dado un umbral $c > 0$, el *recorte por norma global* sustituye cada $g^{(k)}$ por

$$\hat{g}^{(k)} = \alpha\, g^{(k)}, \qquad \alpha = \min\Big(1,\ \frac{c}{\|g\|_2}\Big),$$

con el convenio $\alpha = 1$ si $g = 0$.

Proposición 4.7 (Propiedades del recorte). *Sea $\hat{g} = \alpha g$ el gradiente recortado.*

(i) $\hat{g}$ tiene la dirección y el sentido de g, y $\|\hat{g}\|_2 = \min(\|g\|_2, c)$.

(ii) Un paso de descenso $\theta \leftarrow \theta - \eta\hat{g}$ cumple $\|\Delta\theta\|_2 \le \eta c$.

(iii) Si $\hat{g} \neq 0$, $\hat{g}$ es una dirección de descenso: $\langle \nabla\mathcal{L}, -\hat{g}\rangle < 0$ cuando $\hat{g}$ se calcula con el gradiente exacto.

(iv) Recortar cada tensor por separado con el mismo umbral conserva, en general, la dirección de cada $g^{(k)}$ pero no la de g.

Demostración. (i) $\alpha > 0$, de modo que $\hat{g}$ es un múltiplo positivo de g. Si $\|g\| \le c$, $\alpha = 1$ y $\|\hat{g}\| = \|g\|$; si $\|g\| > c$, $\|\hat{g}\| = c\|g\|/\|g\| = c$. (ii) $\|\Delta\theta\| = \eta\|\hat{g}\| \le \eta c$ por (i). (iii) $\langle g, -\alpha g\rangle = -\alpha\|g\|^2 < 0$. (iv) Con factores distintos $\alpha_k = \min(1, c/\|g^{(k)}\|)$, el vector concatenado $(\alpha_1 g^{(1)}, \dots, \alpha_K g^{(K)})$ es proporcional a g solo si todos los α_k con $g^{(k)} \neq 0$ coinciden. El ejercicio 4.5 da un contraejemplo. ∎

Ejemplo 4.8. Sea $g = (6, 8)$ y $c = 5$. Como $\|g\| = 10$, el factor es $\alpha = 1/2$ y $\hat{g} = (3, 4)$, con norma 5 y la misma dirección (0,6; 0,8). Con $c = 20$, $\alpha = 1$ y el gradiente no cambia.

El recorte lo propusieron Pascanu, Mikolov y Bengio [18, §3.2] para redes recurrentes, donde el corolario 1.6 permite gradientes que explotan; en un acantilado de la superficie de pérdida [9, §8.2.4], el gradiente apunta en una dirección razonable pero tiene una longitud desproporcionada, y el recorte conserva lo primero y corrige lo segundo. En PyTorch se aplica con `torch.nn.utils.clip_grad_norm_` entre `backward()` y el paso del optimizador.

Observación 4.9 (Conexión con los LLM). El recorte por norma global es parte de la receta de preentrenamiento de los modelos abiertos; LLaMA, por ejemplo, usa umbral $c = 1$ [22]. La norma del gradiente antes de recortar, que `clip_grad_norm_` devuelve, es además una magnitud de diagnóstico que conviene registrar en cada paso (tema 6). Con Adam (módulo 7), el gradiente recortado es el que entra en los momentos m_t y v_t.

Lecturas recomendadas para este tema

- Goodfellow, Bengio y Courville [9], §7.12 «Dropout», §8.2.4 «Cliffs and Exploding Gradients» y §10.11 «Optimization for Long-Term Dependencies».
- Zhang *et al.* [26], §5.6 «Dropout».
- Bishop y Bishop [2], §9.6 «Model Averaging» y §9.6.1 «Dropout».
- Notas de CS231n [3], «Neural Networks Part 2», apartado «Regularization» (dropout invertido).
- Srivastava *et al.* [21]; Pascanu, Mikolov y Bengio [18], §3.2 «Scaling down the gradients».

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Al evaluar una red, un estudiante obtiene resultados distintos en cada ejecución con la misma entrada. ¿Qué ha olvidado? ¿Por qué la media de las activaciones no delata el error?

Ejercicio 4.2 ⋆ Durante el entrenamiento se aplica $h_{\text{drop}} = \gamma\,(d \odot h)$, donde cada d_i vale 0 con probabilidad p y 1 con probabilidad $1 - p$. (a) Determine γ para que $\mathbb{E}[h_{\text{drop},i}] = h_i$. (b) Con $p = 0{,}2$, $h = (1, 3, 0, 2)$ y $d = (1, 0, 1, 1)$, calcule h_{drop}. (c) ¿Por qué se aplica dropout al entrenar y no al evaluar?

Fuente: adaptado de Stanford CS224N [4], invierno de 2021, práctica 3, pregunta 1(b), apartados (i) y (ii).

Ejercicio 4.3 ⋆⋆ Aplique dropout invertido con $p = 0{,}3$ a $h = (5, 5, 5, 5)$ con la máscara $m = (1, 0, 1, 1)$. Compruebe que $\mathbb{E}[\tilde{h}_i] = h_i$ y calcule la media de la salida para esta máscara concreta.

Ejercicio 4.4 ⋆⋆ El gradiente es $g = (6, 8)$. Aplique el recorte por norma global con $c = 5$ y con $c = 20$, y compruebe la proposición 4.7(i).

Ejercicio 4.5 ⋆⋆ Un modelo tiene dos tensores de parámetros con gradientes $g^{(1)} = (1, 2)$ y $g^{(2)} = (2)$. (a) Recorte con norma global máxima $c = 1$. (b) Recorte cada tensor por separado con $c = 1$ y compruebe que el vector resultante no es proporcional a $(1, 2, 2)$.

Ejercicio 4.6 ⋆⋆ Una implementación del recorte multiplica todos los gradientes por $M/(\|g\|_2 + \epsilon)$ cuando $\|g\|_2 \geq M$, con $\epsilon = 10^{-6}$. Con $g^{(1)} = (2, -1)$, $g^{(2)} = (2)$ y $M = 1$: (a) calcule la norma global, el factor y los gradientes recortados; (b) calcule la norma del resultado y explique por qué no es exactamente M.

Fuente: adaptado de Stanford CS336 [5], primavera de 2026, práctica 1 (*basics*), versión 26.0.3, §4.5, problema `gradient_clipping`.

Ejercicio 4.7 ⋆⋆⋆ (a) Demuestre la proposición 4.2 y el corolario 4.3. (b) Para $w = (1, 2, -1)$, $h = (2, 1, 2)$ y $p = 0{,}5$, calcule $\mathbb{E}[y]$ y $\mathrm{Var}(y)$ con el corolario y compruébelo enumerando las ocho máscaras posibles. (c) Explique cómo cambia la varianza de las activaciones de una capa oculta cuando se activa el dropout y por qué crece sin límite cuando $p \to 1$.

Fuente: adaptado de Zhang *et al.* [26], §5.6, ejercicio 3 (varianza de las activaciones con y sin dropout).

Tema 5

Aritmética de coma flotante y precisión mixta

Los modelos de lenguaje se entrenan con números de 16 e incluso de 8 bits. Un formato de coma flotante reparte sus bits entre el exponente, que fija el rango, y la mantisa, que fija la precisión relativa; esa elección determina qué valores desbordan, cuáles se anulan y qué sumandos se pierden al acumular. Este tema define los formatos con la norma IEEE 754 [14], demuestra las cotas de rango y de error de redondeo, estudia la absorción de sumandos pequeños y justifica con ella los tres ingredientes de la precisión mixta de Micikevicius *et al.* [16]. Se sigue además el tutorial de Goldberg [8] y el libro de Higham [13].

5.1 Formatos de coma flotante

Definición 5.1 (Formato binario de coma flotante). Sean $e \geq 2$ y $m \geq 1$ enteros. Un *formato binario* con e bits de exponente y m bits de mantisa representa cada número con $1 + e + m$ bits: un signo $s \in \{0, 1\}$, un exponente almacenado $E \in \{0, \ldots, 2^e - 1\}$ y una fracción $f = (f_1 \ldots f_m)_2$. Con el *sesgo* $b = 2^{e-1} - 1$:

(i) si $1 \leq E \leq 2^e - 2$, el número es *normal* y vale $x = (-1)^s \, (1.f_1 \ldots f_m)_2 \cdot 2^{E-b}$;

(ii) si $E = 0$, vale $(-1)^s \, (0.f_1 \ldots f_m)_2 \cdot 2^{1-b}$: el cero si $f = 0$ y un número *subnormal* si $f \neq 0$;

(iii) si $E = 2^e - 1$, representa $\pm\infty$ si $f = 0$ y `NaN` (*not a number*) si $f \neq 0$.

Los formatos binary32 (fp32, $e = 8$, $m = 23$) y binary16 (fp16, $e = 5$, $m = 10$) son los de la norma [14]. El formato bfloat16 (bf16, $e = 8$, $m = 7$) no forma parte de ella; se obtiene conservando los 16 bits más significativos de un fp32.

Proposición 5.2 (Rango y precisión). *En el formato de la definición 5.1:*

(i) el mayor número finito es $x_{\max} = (2 - 2^{-m})\, 2^b$*;*

(ii) el menor normal positivo es $x_{\min} = 2^{1-b}$*, y el menor subnormal positivo,* 2^{1-b-m}*;*

(iii) en cada intervalo $[2^k, 2^{k+1})$ *con* $1 - b \leq k \leq b$*, los números representables están equiespaciados con separación* 2^{k-m}*;*

(iv) el épsilon de máquina*, distancia de 1 al siguiente número representable, es* $\varepsilon = 2^{-m}$*.*

Demostración. (i) El mayor exponente normal es $E = 2^e - 2$, es decir, $E - b = 2^e - 2 - (2^{e-1} - 1) = 2^{e-1} - 1 = b$; la mayor mantisa es $(1.1 \ldots 1)_2 = 2 - 2^{-m}$. (ii) El menor exponente normal es $E = 1$, con $E - b = 1 - b$ y mantisa 1. El menor subnormal tiene $f = (0 \ldots 01)_2 = 2^{-m}$. (iii) Fijado el exponente $k = E - b$, los 2^m valores de f dan los números $(1 + j2^{-m})2^k$, $j = 0, \ldots, 2^m - 1$, separados por 2^{k-m}. (iv) Es (iii) con $k = 0$. ∎

Formato	e	m	$x_{\max}$	$x_{\min}$ (normal)	$\varepsilon = 2^{-m}$	Cifras decimales
fp32	8	23	$3{,}40 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$1{,}19 \cdot 10^{-7}$	≈ 7
fp16	5	10	65 504	$6{,}10 \cdot 10^{-5}$	$9{,}77 \cdot 10^{-4}$	≈ 3
bf16	8	7	$3{,}39 \cdot 10^{38}$	$1{,}18 \cdot 10^{-38}$	$7{,}81 \cdot 10^{-3}$	≈ 2
fp8 E4M3	4	3	448	$1{,}56 \cdot 10^{-2}$	0,125	≈ 1
fp8 E5M2	5	2	57 344	$6{,}10 \cdot 10^{-5}$	0,25	< 1

Cuadro 5.1: Formatos usados en el entrenamiento de redes. El formato E4M3 no sigue la definición 5.1(iii); véase la observación 5.4.

Ejemplo 5.3 (fp16). Con $e = 5$, $m = 10$: $b = 2^4 - 1 = 15$; $x_{\max} = (2 - 2^{-10})2^{15} = 2^{16} - 2^5 = 65\,504$; $x_{\min} = 2^{-14} = 6{,}10 \cdot 10^{-5}$; menor subnormal $2^{-24} = 5{,}96 \cdot 10^{-8}$; $\varepsilon = 2^{-10} = 9{,}77 \cdot 10^{-4}$.

Observación 5.4 (El formato E4M3). Los dos formatos de 8 bits proceden de Micikevicius *et al.* [17]. E5M2 sigue las reglas IEEE. E4M3 renuncia a los infinitos y reserva para `NaN` un único patrón de bits (exponente y fracción con todos los bits a 1). Así, el exponente $E = 15$ queda disponible para números normales y el máximo es $(1{,}110)_2 \cdot 2^{15-7} = 1{,}75 \cdot 2^8 = 448$, en lugar del valor $1{,}875 \cdot 2^7 = 240$ que darían las reglas IEEE.

El formato bf16 tiene el exponente de fp32 y, por tanto, prácticamente su mismo rango, con una precisión mucho menor. Pasar de fp32 a bf16 equivale a redondear la mantisa a 7 bits. Por su rango, bf16 es el formato habitual para entrenar modelos de lenguaje: casi nunca desborda.

5.2 Redondeo y unidad de redondeo

Definición 5.5 (Redondeo al más próximo). Para $x \in \mathbb{R}$ con $|x| \le x_{\max}$, $\mathrm{fl}(x)$ es el número representable más cercano a x; en caso de empate, el de último bit de mantisa par. La *unidad de redondeo* es $u = \varepsilon/2 = 2^{-m-1}$.

Teorema 5.6 (Error relativo de redondeo). *Si* $x_{\min} \le |x| \le x_{\max}$, *entonces* $\mathrm{fl}(x) = x(1+\delta)$ *con* $|\delta| \le u$.

Demostración. Sea $x > 0$ (el caso negativo es simétrico) y sea k tal que $2^k \le x < 2^{k+1}$. Por la proposición 5.2(iii), x está entre dos números representables consecutivos separados por 2^{k-m} (si x supera el mayor número del intervalo, el siguiente es 2^{k+1}, a la misma distancia). El más cercano dista a lo sumo la mitad, $|\mathrm{fl}(x) - x| \le 2^{k-m-1}$. Dividiendo por $x \ge 2^k$, $|\delta| = |\mathrm{fl}(x) - x|/x \le 2^{-m-1} = u$. ∎

La norma IEEE 754 exige que la suma, la resta, el producto, el cociente y la raíz cuadrada se redondeen correctamente: el resultado calculado es $\mathrm{fl}(x \circ y)$, el redondeo del resultado exacto. Junto con el teorema 5.6, esto da el *modelo estándar* del análisis de errores [13, 8]: $\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta)$ con $|\delta| \le u$, mientras no haya desbordamiento ni subdesbordamiento.

Lema 5.7 (Absorción). *Sea* $x > 0$ *representable con* $2^k \le x < 2^{k+1}$ *y sea* $0 \le y < 2^{k-m-1}$, *la mitad de la separación del intervalo. Entonces* $\mathrm{fl}(x + y) = x$.

Demostración. El siguiente representable mayor que x es $x + 2^{k-m}$. Como $x \le x + y < x + 2^{k-m-1}$, el resultado exacto está más cerca de x que de $x + 2^{k-m}$, y x es el representable más cercano. ∎

Corolario 5.8 (Estancamiento de una suma). *Sea $a > 0$ representable y sea la suma recursiva $s_0 = 0$, $s_{j+1} = \mathrm{fl}(s_j + a)$. Si para algún j se cumple $2^k \le s_j < 2^{k+1}$ con $a < 2^{k-m-1}$, entonces $s_i = s_j$ para todo $i \ge j$. En particular, la suma no supera la menor potencia 2^k que cumple $2^k > a\,2^{m+1}$, más una separación.*

Demostración. La primera afirmación es el lema 5.7 aplicado repetidamente. Para la segunda, la suma es creciente; cuando alcanza el intervalo que empieza en esa potencia 2^k se cumple $a < 2^{k-m-1}$ y queda detenida, y la suma exacta $s_j + a$ que la hizo entrar en él es menor que $2^k + a < 2^k + 2^{k-m}$. ∎

Operación	fp32	fp16	bf16
$300 \cdot 300$	90 000	`inf`	90 112
e^{12}	162 754,8	`inf`	162 816
$1 + 2^{-8}$	1,0039	1,0039	1
$1000 + 1$	1001	1001	1000
0,001 sumado 10^4 veces	10,0004	4	0,5

Cuadro 5.2: Resultados de operaciones sencillas en tres formatos, comprobados con PyTorch.

Ejemplo 5.9 (Estancamiento en fp16 y bf16). En la última fila de la cuadro 5.2, el sumando 0,001 se almacena en fp16 como $a = 0{,}0010004$. Con $m = 10$, $a\,2^{11} = 2{,}049$, y la menor potencia de 2 mayor es 4: en el intervalo $[4, 8)$ la semiseparación es $2^{2-11} = 0{,}00195 > a$ y la suma se detiene en 4. En bf16, $a = 0{,}00099945$ y $a\,2^8 = 0{,}256$, de modo que la suma se detiene en 0,5.

Teorema 5.10 (Error de la suma recursiva). *Sea $\hat{s}_n$ el resultado de sumar $x_1, \dots, x_n$ en orden, $\hat{s}_1 = x_1$, $\hat{s}_k = \mathrm{fl}(\hat{s}_{k-1} + x_k)$, sin desbordamiento ni subdesbordamiento, y sea $(n-1)u < 1$. Entonces*

$$|\hat{s}_n - \sum_{i=1}^{n} x_i| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \qquad \gamma_k = \frac{ku}{1 - ku}.$$

Demostración. Por el modelo estándar, $\hat{s}_k = (\hat{s}_{k-1} + x_k)(1 + \delta_k)$ con $|\delta_k| \le u$. Desarrollando la recurrencia, $\hat{s}_n = \sum_{i=1}^{n} x_i \prod_{j=\max(i,2)}^{n} (1 + \delta_j)$, donde cada producto tiene a lo sumo $n - 1$ factores. Para un producto de k factores, $|\prod_j (1+\delta_j) - 1| \le (1+u)^k - 1 \le e^{ku} - 1 \le \frac{1}{1-ku} - 1 = \gamma_k$, usando $e^t \le 1/(1-t)$ para $0 \le t < 1$ (la primera desigualdad se obtiene desarrollando el producto y acotando cada término por su valor absoluto). Como γ_k crece con k, restando $\sum_i x_i$ y aplicando la desigualdad triangular se obtiene la cota [13]. ∎

El teorema explica la diferencia entre formatos. En fp32, $u = 2^{-24} = 5{,}96 \cdot 10^{-8}$ y, para $n = 10^4$ sumandos de 0,001, la cota es $\gamma_{9999} \cdot 10 = 0{,}006$; el error observado en la cuadro 5.2 es $4 \cdot 10^{-4}$. En fp16, $u = 2^{-11}$ y $(n-1)u \approx 4{,}9 > 1$: la cota no garantiza nada, y la suma se estanca.

5.3 Precisión mixta

Micikevicius *et al.* [16, §3] entrenan en fp16 sin perder precisión con tres técnicas.

Copia maestra de los pesos en fp32 (§3.1). El optimizador actualiza una copia fp32 de los pesos; la copia fp16 solo se usa en las pasadas hacia delante y hacia atrás. Por el lema 5.7, si un peso $w \in [2^k, 2^{k+1})$ recibe una actualización $|\Delta w| < 2^{k-m-1}$, la suma $w + \Delta w$ redondeada devuelve w. Como $w < 2^{k+1}$, esto ocurre seguro si $|w|/|\Delta w| > 2^{m+2}$ y puede ocurrir desde 2^{m+1}; en fp16 este último cociente es $2^{11} = 2048$, el que señala el artículo.

Escalado de la pérdida (§3.2). Muchos gradientes de activaciones son menores que el menor subnormal de fp16, 2^{-24}, y se anulan. Se multiplica la pérdida por un factor S, se retropropaga y se dividen los gradientes de los pesos por S antes de la actualización, que se omite si aparece un `inf` o un `NaN`. La elección $S = 2^j$ es conveniente por la proposición siguiente.

Proposición 5.11. *Si x y $2^j x$ están en el rango normal, $\mathrm{fl}(2^j x) = 2^j\,\mathrm{fl}(x)$. En particular, multiplicar o dividir un número representable por 2^j es exacto en ausencia de desbordamiento.*

Demostración. Multiplicar por 2^j suma j al exponente y conserva la mantisa, de modo que biyecta los representables de $[2^k, 2^{k+1})$ con los de $[2^{k+j}, 2^{k+j+1})$ y multiplica todas las distancias por 2^j. El más cercano a $2^j x$ es, por tanto, 2^j veces el más cercano a x. ∎

Acumulación en fp32 (§3.3). Los productos escalares de las multiplicaciones de matrices acumulan las sumas parciales en fp32 y solo convierten el resultado a fp16; las reducciones grandes (sumas de la normalización y de la softmax) también se hacen en fp32. El teorema 5.10 justifica esta práctica.

Observación 5.12 (bf16 y fp8). Con bf16 no hace falta escalar la pérdida, porque su rango es el de fp32, pero la copia maestra y la acumulación en fp32 siguen siendo necesarias, porque su precisión es aún menor que la de fp16. Los formatos fp8 se usan en las GPU recientes para las multiplicaciones de matrices, E4M3 en la pasada hacia delante y E5M2 para los gradientes, siempre con factores de escala por tensor [17]; el módulo 17 los estudia con el hardware y el módulo 19 trata la cuantización para inferencia. Anthropic, Google y xAI no han hecho pública la precisión con que entrenan Claude, Gemini y Grok.

Lecturas recomendadas para este tema

- IEEE 754-2019 [14], cláusulas sobre formatos binarios y redondeo.
- Goldberg [8], partes dedicadas al error de redondeo y a la norma IEEE.
- Higham [13], capítulos sobre aritmética de coma flotante y sobre sumación.
- Micikevicius *et al.* [16], §3 «Implementation»; y [17] para los formatos de 8 bits.
- Stanford CS336 [6], práctica 2 (*systems*), §2.1 sobre precisión mixta.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ ¿En qué formato es más arriesgado almacenar una activación de valor 70 000: fp16, bf16 o fp32? ¿En cuál se pierde más precisión al almacenar 1,003?

Ejercicio 5.2 ⋆⋆ Deduzca el máximo, el menor normal y el épsilon de fp16 ($e = 5$, $m = 10$) y de bf16 ($e = 8$, $m = 7$) a partir de la proposición 5.2.

Ejercicio 5.3 ⋆⋆ (a) Calcule el sesgo, el máximo, el menor normal y el épsilon de E5M2 con las reglas de la definición 5.1. (b) Calcule los mismos valores para E4M3 si se siguieran esas reglas, y explique por qué su máximo real es 448. (c) ¿Cuál de los dos formatos es más adecuado para gradientes y cuál para activaciones?

Ejercicio 5.4 ⋆⋆ En bf16 los números de $[512, 1024)$ están separados por $2^9 \cdot 2^{-7} = 4$. (a) Calcule $1000+1$ y $1000+4$ en bf16. (b) Calcule $1000+1$ en fp16, con separación $2^9 \cdot 2^{-10} = 0{,}5$. (c) ¿A partir de qué tamaño se pierde un sumando al añadirlo a 1000 en bf16?

Ejercicio 5.5 ⋆⋆ Se recomienda convertir la entrada de RMSNorm a fp32 antes de elevarla al cuadrado. Compruébelo con $x = (300, 400)$ en fp16, $g = \mathbf{1}$ y $\epsilon = 10^{-5}$: (a) calcule $\mathrm{RMSNorm}(x)$ con todas las operaciones en fp16; (b) calcúlelo en fp32 y convierta el resultado a fp16.

Fuente: adaptado de Stanford CS336 [5], primavera de 2026, práctica 1 (*basics*), versión 26.0.3, §3.4.1, problema `rmsnorm`.

Ejercicio 5.6 ⋆⋆⋆ (a) Demuestre el teorema 5.6. (b) Demuestre el lema 5.7. (c) Deduzca que, en fp16, una actualización Δw de un peso $w > 0$ se pierde con seguridad si $w/|\Delta w| > 4096$, y dé un ejemplo con $w = 1$ en el que se pierda con $w/|\Delta w|$ ligeramente superior a 2048.

Ejercicio 5.7 ⋆⋆⋆ Se suma 0,01 mil veces partiendo de $s = 0$ en cuatro variantes: (i) todo en fp32; (ii) todo en fp16; (iii) acumulador fp32 y sumando fp16; (iv) como (iii), convirtiendo antes el sumando a fp32. (a) Explique por qué 0,01 vale en fp16 exactamente $1311 \cdot 2^{-17} = 0{,}0100021\ldots$ (b) Calcule cuánto aumenta s en cada suma en fp16 cuando $s \in [4, 8)$ y cuando $s \in [8, 16)$. (c) Explique los resultados medidos: (i) 10,0001; (ii) 9,953; (iii) y (iv) 10,0021.

Fuente: adaptado de Stanford CS336 [6], primavera de 2026, práctica 2 (*systems*), versión 26.1.3, §2.1.5, problema `mixed_precision_accumulation`.

Tema 6

Estabilidad numérica de softmax y entropía cruzada

La capa de salida de un modelo de lenguaje calcula una softmax sobre decenas de miles de logits, y la pérdida de entrenamiento es su entropía cruzada (módulo 1). Ambas contienen exponenciales, que en coma flotante desbordan o se anulan con argumentos moderados. Este tema determina esos límites, introduce la función log-sum-exp y su evaluación estable, deduce de ella la forma correcta de calcular la entropía cruzada a partir de los logits y termina con la propagación de `NaN` y un procedimiento para diagnosticar una pérdida que deja de ser finita. Se sigue [9, §4.1], [26, §4.5.2] y las prácticas de CS336 [5].

6.1 Límites de la exponencial

Proposición 6.1. *En un formato con mayor número finito $x_{\max}$ y menor normal $x_{\min}$, el valor exacto e^t es finito y normal si y solo si $\ln x_{\min} \le t \le \ln x_{\max}$. Los intervalos son, aproximadamente,*

$$\textit{fp32 y bf16: } [-87{,}3;\ 88{,}7], \qquad \textit{fp16: } [-9{,}70;\ 11{,}09].$$

Demostración. La exponencial es estrictamente creciente, de modo que $x_{\min} \le e^t \le x_{\max}$ equivale a tomar logaritmos en ambas desigualdades. Con la cuadro 5.1: $\ln(3{,}40\cdot 10^{38}) = 88{,}72$, $\ln 2^{-126} = -87{,}34$, $\ln 65\,504 = 11{,}09$ y $\ln 2^{-14} = -9{,}70$; el máximo de bf16 da $\ln(3{,}39\cdot 10^{38}) = 88{,}72$ con tres decimales. ∎

Por debajo del límite inferior, el resultado es subnormal, con menos bits de precisión, y algo más abajo se anula. Goodfellow *et al.* [9, §4.1] describen las dos formas en que falla la softmax ingenua $e^{z_i}/\sum_j e^{z_j}$: si algún logit es grande, el numerador y el denominador desbordan y el cociente ∞/∞ es `NaN`; si todos son muy negativos, ambos se anulan y $0/0$ es `NaN`.

6.2 La función log-sum-exp

Definición 6.2 (Log-sum-exp). Para $z \in \mathbb{R}^n$, $\mathrm{LSE}(z) = \ln \sum_{i=1}^{n} e^{z_i}$. La softmax del módulo 1 se escribe $\mathrm{softmax}(z)_i = e^{z_i - \mathrm{LSE}(z)}$.

Proposición 6.3 (Propiedades de log-sum-exp). *Sea $z \in \mathbb{R}^n$ y $c \in \mathbb{R}$.*

(i) $\mathrm{LSE}(z + c\mathbf{1}) = \mathrm{LSE}(z) + c$ *y* $\mathrm{softmax}(z + c\mathbf{1}) = \mathrm{softmax}(z)$.

(ii) $\max_i z_i \le \mathrm{LSE}(z) \le \max_i z_i + \ln n$*; la cota superior se alcanza si y solo si todas las componentes son iguales.*

(iii) $\nabla \mathrm{LSE}(z) = \mathrm{softmax}(z)$.

(iv) $\nabla^2 \operatorname{LSE}(z) = \operatorname{diag}(p) - pp^\top$ *con* $p = \operatorname{softmax}(z)$*, que es semidefinida positiva; en consecuencia,* LSE *es convexa.*

Demostración. (i) $\sum_i e^{z_i+c} = e^c \sum_i e^{z_i}$; se toma el logaritmo. Para la softmax, $e^{z_i+c-\operatorname{LSE}(z)-c} = e^{z_i-\operatorname{LSE}(z)}$. (iii) $\partial \operatorname{LSE}/\partial z_k = e^{z_k}/\sum_i e^{z_i} = \operatorname{softmax}(z)_k$. Los apartados (ii) y (iv) son los ejercicios 6.6 y 6.7. ■

Proposición 6.4 (Evaluación estable). *Sea* $c = \max_i z_i$*. Entonces*

$$\operatorname{LSE}(z) = c + \ln S, \qquad \operatorname{softmax}(z)_i = \frac{e^{z_i-c}}{S}, \qquad S = \sum_{i=1}^{n} e^{z_i-c} \in [1, n].$$

En esta forma ninguna exponencial desborda, porque todos sus argumentos son ≤ 0*, y el denominador nunca se anula, porque contiene el término* $e^0 = 1$*.*

Demostración. Las fórmulas son la proposición 6.3(i) con la constante $-c$. Cada $z_i - c \le 0$ da $e^{z_i-c} \in (0, 1]$, y el término con $z_i = c$ vale exactamente 1; por tanto $1 \le S \le n$, y $\ln S \in [0, \ln n]$ se calcula sin problemas. ■

Ejemplo 6.5. Para $z = (1000, 999, 998)$, la evaluación ingenua da $e^{1000} = \infty$ en fp32 y la softmax es `NaN`. Con $c = 1000$, $S = 1 + e^{-1} + e^{-2} = 1{,}503$, $\operatorname{LSE}(z) = 1000 + \ln 1{,}503 = 1000{,}408$ y $\operatorname{softmax}(z) = (0{,}665;\ 0{,}245;\ 0{,}090)$, la misma que para $z = (2, 1, 0)$ en el módulo 1, de acuerdo con la proposición 6.3(i). En fp16 basta $z = (12, 10, 0)$ para desbordar, porque $e^{12} = 162\,755 > 65\,504$.

6.3 Entropía cruzada a partir de los logits

Proposición 6.6 (Log-softmax y entropía cruzada). *Sea* $z \in \mathbb{R}^n$ *el vector de logits y* $y \in \{1, \dots, n\}$ *la clase correcta. Entonces*

$$\ln \operatorname{softmax}(z)_i = z_i - \operatorname{LSE}(z), \qquad \ell(z, y) = -\ln \operatorname{softmax}(z)_y = \operatorname{LSE}(z) - z_y \ge 0,$$

$$\nabla_z \ell = \operatorname{softmax}(z) - e_y,$$

donde e_y *es el vector canónico.*

Demostración. La primera igualdad es el logaritmo de la definición 6.2; la segunda, su caso $i = y$, y no es negativa por la proposición 6.3(ii), ya que $\operatorname{LSE}(z) \ge \max_i z_i \ge z_y$. El gradiente se sigue de la proposición 6.3(iii) y de $\nabla_z z_y = e_y$. ■

La fórmula $\ell = \operatorname{LSE}(z) - z_y$, calculada con la proposición 6.4, no evalúa nunca el logaritmo de un número que pueda haberse anulado. Por eso las funciones de pérdida de las bibliotecas, como `F.cross_entropy` de PyTorch, reciben logits y no probabilidades [26, §4.5.2], y la composición `torch.log(torch.softmax(z))` debe evitarse.

Ejemplo 6.7. En fp16, con $z = (0, -20)$, la softmax ingenua da $e^{-20}/(1 + e^{-20})$ con $e^{-20} = 2{,}1 \cdot 10^{-9}$, menor que el menor subnormal $2^{-24} = 6 \cdot 10^{-8}$: se redondea a 0, y $\ln 0 = -\infty$. La fórmula de la proposición 6.6 da $\ln \operatorname{softmax}(z)_2 = -20 - \ln(1 + e^{-20}) = -20{,}000$, el valor correcto.

6.4 Propagación de NaN y diagnóstico

La norma IEEE 754 [14] define como `NaN` los resultados de las operaciones inválidas $\infty - \infty$, $0 \cdot \infty$, $0/0$, ∞/∞ y $\sqrt{x}$ con $x < 0$, y establece que toda operación aritmética con un operando `NaN` da `NaN`; además, toda comparación con `NaN` es falsa. En particular, $\texttt{NaN} \cdot 0 = \texttt{NaN}$: multiplicar por cero no limpia el valor.

Observación 6.8 (Contagio). Si la pérdida de un paso es `NaN`, la retropropagación multiplica ese valor por todos los jacobianos locales, de modo que el gradiente de todo parámetro del que depende la pérdida es `NaN`, y la actualización convierte esos parámetros en `NaN`. A partir de ese paso la red ya no produce números. Por eso el diagnóstico debe hacerse en el primer paso en que aparece un valor no finito.

Síntoma	Causa probable	Remedio
La pérdida crece varios órdenes de magnitud y luego da `NaN`	tasa de aprendizaje alta; gradientes que explotan (tema 1)	bajar la tasa, calentamiento, recorte del gradiente (tema 4)
`inf` repentino en fp16	desbordamiento por encima de 65 504	bf16, o escalado dinámico de la pérdida (tema 5)
`NaN` en una softmax o un logaritmo	exponencial desbordada, $\ln 0$	proposición 6.4; entropía cruzada con logits
`NaN` en una normalización	división por $\mathrm{RMS}(x) = 0$	$\epsilon > 0$ dentro de la raíz (tema 3)
Picos aislados de la pérdida	lote anómalo	recorte, descartar el lote, volver a un punto de control

Cuadro 6.1: Diagnóstico de una pérdida no finita.

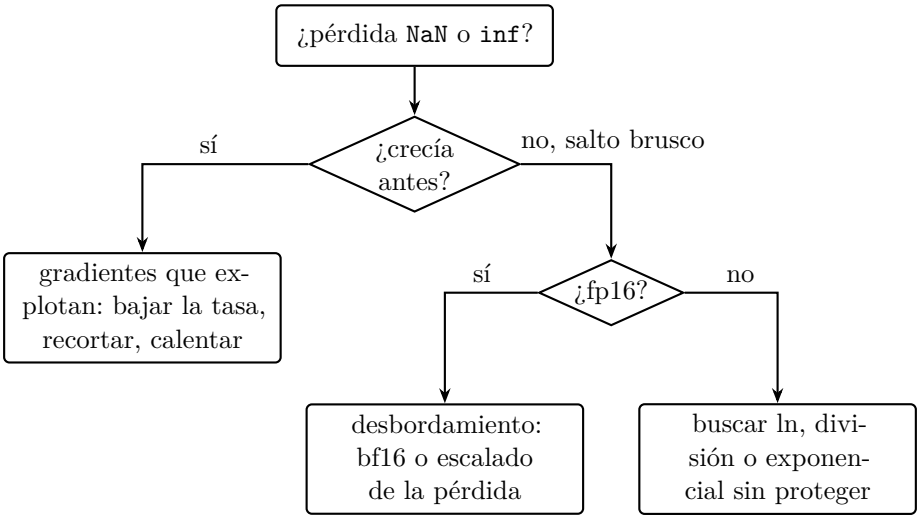


Figura 6.1: Árbol de decisión para una pérdida no finita. En todos los casos conviene registrar la norma del gradiente por capa y los histogramas de activaciones.

Observación 6.9 (Lo que se observa en la práctica). En la práctica del módulo se provoca el fallo a propósito: una red residual sin normalizar entrenada con tasa de aprendizaje 10 pasa de una pérdida de 2,6 a 10^{30} y llega a `NaN` en el paso 24. Con recorte del gradiente a norma 1, la pérdida se mantiene finita durante los 60 pasos.

Lecturas recomendadas para este tema

- Goodfellow, Bengio y Courville [9], §4.1 «Overflow and Underflow».
- Zhang *et al.* [26], §4.5.2 «Softmax Revisited».
- Stanford CS336 [5], práctica 1 (*basics*), problema `softmax` (§3.4.4) y entropía cruzada estable.
- IEEE 754-2019 [14], tratamiento de las excepciones y de los valores `NaN`.

Ejercicios del tema 6

Ejercicio 6.1 ⋆ Indique la causa más probable y un remedio para cada caso: (a) en fp16 la pérdida es normal y en un paso concreto se vuelve `inf`, sin crecimiento previo; (b) la pérdida pasa de 2,3 a 10^6 en pocos pasos y luego a `NaN`; (c) la pérdida es `NaN` desde el primer paso en una red que usa `torch.log(torch.softmax(z))`.

Ejercicio 6.2 ⋆⋆ Calcule $\mathrm{LSE}(1000, 999, 998)$ y la softmax correspondiente con tres decimales, y compruebe las cotas de la proposición 6.3(ii).

Ejercicio 6.3 ⋆⋆ Sean las puntuaciones $v = (100, 101, 103)$ en fp32. (a) ¿Qué devuelve la softmax ingenua y por qué? (b) Demuestre que $\mathrm{softmax}(v - c\mathbf{1}) = \mathrm{softmax}(v)$ para toda constante c. (c) Calcule la softmax con $c = \max_i v_i$.

Fuente: adaptado de Stanford CS336 [5], primavera de 2026, práctica 1 (*basics*), versión 26.0.3, §3.4.4, problema `softmax`.

Ejercicio 6.4 ⋆⋆ Calcule el menor y el mayor argumento t para los que e^t no se desborda ni da un resultado subnormal en fp32, fp16 y bf16. ¿Qué formato limita más los logits de una softmax ingenua?

Fuente: adaptado de Zhang *et al.* [26], §4.5, ejercicio 1 (se omiten los formatos FP64, TF32 e INT8).

Ejercicio 6.5 ⋆⋆ (a) Para los logits $z = (2, 1, 0)$ y la clase correcta $y = 3$, calcule la entropía cruzada $\ell(z, y)$ y su gradiente con la proposición 6.6. (b) En fp16, con $z = (0, -20)$, calcule $\ln \mathrm{softmax}(z)_2$ de forma ingenua y de forma estable.

Ejercicio 6.6 ⋆⋆⋆ Demuestre la proposición 6.3(ii): $\max_i z_i \le \mathrm{LSE}(z) \le \max_i z_i + \ln n$, con igualdad en la cota superior si y solo si todas las componentes de z son iguales. Compruébelo con $z = (3, 3)$ y con $z = (2, 1, 0)$.

Ejercicio 6.7 ⋆⋆⋆ (a) Demuestre que $\nabla^2 \mathrm{LSE}(z) = \mathrm{diag}(p) - pp^\top$ con $p = \mathrm{softmax}(z)$. (b) Demuestre que, para todo $v \in \mathbb{R}^n$, $v^\top(\mathrm{diag}(p) - pp^\top)v = \sum_i p_i (v_i - \bar{v})^2 \ge 0$, con $\bar{v} = \sum_i p_i v_i$, y concluya que LSE y la entropía cruzada $\ell(\cdot, y)$ son convexas en los logits. (c) Compruebe que $\mathbf{1}$ está en el núcleo del hessiano y relaciónelo con la proposición 6.3(i).

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; las potencias de 2 de los formatos numéricos son exactas. Los valores en coma flotante se han comprobado con PyTorch.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. red plana de 24 capas; factor por capa $1/\sqrt{3}$ con la inicialización por defecto (ejemplo 1.12).

Planteamiento. por la proposición 1.2, el gradiente de los pesos de la capa ℓ contiene el producto de los $L - \ell$ jacobianos que tiene encima.

Desarrollo.

1. El gradiente de la capa 1 atraviesa los jacobianos $J_2, \ldots, J_{24}$: 23 factores.
2. Si cada uno escala por $1/\sqrt{3} = 0{,}5774$, el producto es $0{,}5774^{23} = 3{,}3 \cdot 10^{-6}$.
3. El gradiente de la capa 24 no atraviesa ningún jacobiano: factor 1.
4. Con la misma tasa de aprendizaje, el paso de la capa 1 es unas $3 \cdot 10^5$ veces menor que el de la última: sus pesos apenas cambian.

Resultado. **la capa 1 recibe un gradiente del orden de $3 \cdot 10^{-6}$ veces el de la última y queda prácticamente congelada**, aunque la última aprenda con normalidad.

Comprobación. la práctica del módulo mide un cociente de $1{,}4 \cdot 10^{-5}$ entre las capas 1 y 24, del mismo orden de magnitud.

Solución del ejercicio 1.2

Datos. 24 capas; factores 0,5 y 1,1; umbral 2^{-24}.

Planteamiento. el factor total es el producto de los factores por capa (ejemplo 1.3).

Desarrollo.

1. (a) $0{,}5^{24} = 1/2^{24} = 1/16\,777\,216 = 5{,}960 \cdot 10^{-8}$.
2. (b) $1{,}1^2 = 1{,}21$; $1{,}1^4 = 1{,}4641$; $1{,}1^8 = 2{,}1436$; $1{,}1^{16} = 4{,}5950$; $1{,}1^{24} = 1{,}1^{16} \cdot 1{,}1^8 = 4{,}5950 \cdot 2{,}1436 = 9{,}850$.
3. (c) $0{,}5^L < 2^{-24} \iff 2^{-L} < 2^{-24} \iff L > 24$. Con $L = 24$ el factor es exactamente 2^{-24}; hace falta $L = 25$.

Resultado. **(a)** $5{,}96 \cdot 10^{-8}$**; (b)** $9{,}85$**; (c) 25 capas.**

Comprobación. por logaritmos, $1{,}1^{24} = e^{24 \ln 1{,}1} = e^{24 \cdot 0{,}09531} = e^{2{,}287} = 9{,}85$.

Solución del ejercicio 1.3

Datos. $\tanh'(t) = 1 - \tanh^2(t)$; 5 capas, $w_\ell = 1$, preactivaciones 2; $s'(t) \le 1/4$.

Planteamiento. el jacobiano escalar de cada capa es $f'(z_\ell)\, w_\ell$ (definición 1.1).

Desarrollo.

1. (a) $\tanh 0 = 0 \Rightarrow \tanh'(0) = 1$; $\tanh 1 = 0{,}7616$, $\tanh^2 1 = 0{,}5800 \Rightarrow \tanh'(1) = 0{,}4200$.
2. $\tanh 2 = 0{,}9640$, $\tanh^2 2 = 0{,}9293 \Rightarrow \tanh'(2) = 0{,}07065$; $\tanh 3 = 0{,}9951$, $\tanh^2 3 = 0{,}9901 \Rightarrow \tanh'(3) = 0{,}009866$.
3. (b) Cada capa aporta $0{,}07065 \cdot 1$; el producto es $0{,}07065^5 = 1{,}760 \cdot 10^{-6}$.
4. (c) La sigmoide alcanza su derivada máxima $1/4$ en $t = 0$; en el mejor caso, $(1/4)^5 = 1/1024 = 9{,}766 \cdot 10^{-4}$.

Resultado. **(a)** 1**;** $0{,}420$**;** $0{,}0707$**;** $0{,}0099$**. (b)** $1{,}76 \cdot 10^{-6}$**. (c)** $9{,}77 \cdot 10^{-4}$**.**

Comprobación. el resultado (c) coincide con γ^L del teorema 1.5 con $\gamma = 1/4$, $\|W\| = 1$ y $L = 5$: incluso en el mejor caso, la sigmoide reduce el gradiente.

Solución del ejercicio 1.4

Datos. $n = 64$; varianza inicial 1; $\sigma^2 = 1/(3n)$ por defecto.

Planteamiento. corolario 1.10 ($\sigma^2 = 1/n$), proposición 1.11 ($\sigma^2 = 2/n$) y factor $n\sigma^2$ por capa.

Desarrollo.

1. (a) Red lineal: $\sigma^2 = 1/64 = 0{,}015625$, $\sigma = 1/8 = 0{,}125$.
2. Red ReLU: $\sigma^2 = 2/64 = 0{,}03125$, $\sigma = \sqrt{0{,}03125} = 0{,}1768$.
3. (b) $n\sigma^2 = 64 \cdot 1/(3 \cdot 64) = 1/3$.
4. Tras 10 capas: $(1/3)^{10} = 1/59\,049 = 1{,}694 \cdot 10^{-5}$.

Resultado. **(a)** $\sigma = 0{,}125$ **(Xavier) y** $\sigma \approx 0{,}177$ **(He); (b) factor** $1/3$ **y varianza** $1{,}69 \cdot 10^{-5}$ **tras 10 capas.**

Comprobación. He multiplica la varianza de Xavier por 2 y, por tanto, la desviación típica por $\sqrt{2}$: $0{,}125 \cdot 1{,}414 = 0{,}1768$.

Solución del ejercicio 1.5

Datos. $A, B \in \mathbb{R}^{d \times d}$; $s_{\min} = 0{,}9$, $s_{\max} = 1{,}1$, $L = 24$.

Planteamiento. los valores singulares de una matriz C son los valores $\|Cv\|$ para sus vectores singulares derechos v unitarios, de modo que todos están entre $\sigma_{\min}(C)$ y $\|C\|_2$.

Desarrollo.

1. (a) Sea σ un valor singular de AB con vector singular derecho v, $\|v\| = 1$. Entonces $\sigma = \|ABv\|$.
2. $\|ABv\| \le \|A\|_2 \|Bv\| \le \|A\|_2 \|B\|_2$, y $\|ABv\| \ge \sigma_{\min}(A)\|Bv\| \ge \sigma_{\min}(A)\sigma_{\min}(B)$ (lema 1.4).

3. (b) Por inducción sobre el número de factores, $\sigma_{\min}(J_1^\top \cdots J_L^\top) \geq \prod_\ell \sigma_{\min}(J_\ell) \geq s_{\min}^L$ y $\|J_1^\top \cdots J_L^\top\|_2 \leq s_{\max}^L$; el cociente del teorema 1.5 queda entre ambos.
4. $0{,}9^{24} = 0{,}07977$ y $1{,}1^{24} = 9{,}850$.
5. (c) El intervalo [0,080; 9,85] es amplio aunque cada jacobiano esté a un 10 % de una isometría: el número de condición del producto puede llegar a $(1{,}1/0{,}9)^{24} = 123{,}5$. Para un gradiente bien condicionado, todos los valores singulares de cada jacobiano deben estar muy próximos a 1.

Resultado. **los valores singulares de AB están en $[\sigma_{\min}(A)\sigma_{\min}(B),\ \|A\|_2\|B\|_2]$; con 24 capas, el cociente está en** [0,080; 9,85]**.**

Comprobación. con $A = \mathrm{diag}(2, 1)$ y $B = \mathrm{diag}(1, 3)$, $AB = \mathrm{diag}(2, 3)$ tiene valores singulares 2 y 3, dentro de $[1 \cdot 1,\ 2 \cdot 3] = [1, 6]$.

Solución del ejercicio 1.6

Datos. $M = \sum_i \lambda_i v_i v_i^\top$ con $v_i^\top v_j = \delta_{ij}$; $M = \left(\begin{smallmatrix} 0{,}85 & 0{,}35 \\ 0{,}35 & 0{,}85 \end{smallmatrix}\right)$, $x = (1, 0)$.

Planteamiento. descomposición espectral (módulo 3) y desarrollo de x en la base de autovectores.

Desarrollo.

1. (a) Por inducción: si $M^k = \sum_i \lambda_i^k v_i v_i^\top$, entonces $M^{k+1} = \sum_{i,j} \lambda_i^k \lambda_j v_i (v_i^\top v_j) v_j^\top = \sum_i \lambda_i^{k+1} v_i v_i^\top$. Así $M^k v_i = \lambda_i^k v_i$.
2. (b) Sea $x = \sum_i c_i v_i$ con $c_1 \neq 0$. Entonces $M^k x = \lambda_1^k \big(c_1 v_1 + \sum_{i \geq 2} c_i (\lambda_i/\lambda_1)^k v_i\big)$.
3. Como $|\lambda_i/\lambda_1| < 1$ para $i \geq 2$, los términos de la suma tienden a 0, y $M^k x / \|M^k x\| \to \mathrm{signo}(c_1 \lambda_1^k)\, v_1$.
4. (c) $x = (1, 0)$ tiene $c_1 = c_2 = 1/\sqrt{2}$ respecto de $v_1 = (1, 1)/\sqrt{2}$ y $v_2 = (1, -1)/\sqrt{2}$. Para $k = 1$, $Mx = (0{,}85;\ 0{,}35)$, de dirección (0,9247; 0,3807).
5. Para $k = 10$, $M^{10}x = (3{,}0964;\ 3{,}0954)$ (ejemplo 1.7), de norma 4,3782 y dirección (0,70722; 0,70700), frente a $v_1 = (0{,}70711;\ 0{,}70711)$.
6. (d) En una red recurrente lineal el gradiente contiene potencias $(W^\top)^k$. Su componente sobre el autovector dominante escala como λ_1^k, que explota si $|\lambda_1| > 1$ y se desvanece si $|\lambda_1| < 1$; además, la dirección del gradiente queda dominada por v_1, con independencia de la señal de error.

Resultado. M^k **tiene autovalores λ_i^k y $M^k x$ se alinea con v_1; con $k = 10$ la dirección coincide con v_1 en tres decimales.**

Comprobación. $1{,}2^{10} = 6{,}1917$ y $\frac{1}{2}(6{,}1917 + 0{,}00098) = 3{,}0964$, $\frac{1}{2}(6{,}1917 - 0{,}00098) = 3{,}0954$.

A.2 Tema 2

Solución del ejercicio 2.1

Planteamiento. separar lo que se puede representar de lo que se puede aprender con datos y recursos finitos.

Desarrollo.

1. Razón estadística: una clase mayor contiene funciones que se ajustan a los datos de entrenamiento sin generalizar; con datos finitos, aumentar la capacidad puede aumentar el error de prueba (módulo 8).
2. Razón computacional: el coste de cálculo y de memoria de cada paso crece linealmente con la profundidad, y la pasada hacia delante es secuencial a lo largo de las capas, lo que aumenta la latencia.
3. Además, que la clase contenga una buena función no garantiza que el optimizador la encuentre; el flujo residual crece con la profundidad (proposición 2.8).

Resultado. **la inclusión de clases garantiza que la profundidad no empeora la capacidad de representación, pero no la generalización ni el coste.**

Solución del ejercicio 2.2

Datos. 24 bloques; $J_\ell = \pm 0{,}01$; red plana con factor 0,5.

Planteamiento. en el caso escalar, el factor residual es $\prod_\ell (1 + J_\ell)$ (teorema 2.4).

Desarrollo.

1. (a) $1{,}01^2 = 1{,}0201$; $1{,}01^4 = 1{,}0406$; $1{,}01^8 = 1{,}0829$; $1{,}01^{16} = 1{,}1726$; $1{,}01^{24} = 1{,}1726 \cdot 1{,}0829 = 1{,}270$.
2. (b) $0{,}99^{24}$: $0{,}99^2 = 0{,}9801$; $0{,}99^4 = 0{,}9606$; $0{,}99^8 = 0{,}9227$; $0{,}99^{16} = 0{,}8515$; $0{,}99^{24} = 0{,}8515 \cdot 0{,}9227 = 0{,}7857$.
3. (c) Red plana: $0{,}5^{24} = 5{,}96 \cdot 10^{-8}$. Red residual con $J_\ell = -0{,}5$: $(1 - 0{,}5)^{24} = 5{,}96 \cdot 10^{-8}$, idéntico.

Resultado. **(a)** 1,27**; (b)** 0,786**; (c) la red residual solo ayuda si las ramas son pequeñas; si todas se oponen a la identidad con $J_\ell = -0{,}5$, el factor es el mismo que en la red plana.**

Comprobación. $e^{\pm 0{,}24} = 1{,}271$ y 0,787, de acuerdo con $(1 \pm x)^n \approx e^{\pm nx}$.

Solución del ejercicio 2.3

Datos. $J_1 = \begin{pmatrix} 0{,}2 & 0 \\ 0{,}1 & -0{,}1 \end{pmatrix}$, $J_2 = \begin{pmatrix} -0{,}1 & 0{,}3 \\ 0 & 0{,}2 \end{pmatrix}$.

Planteamiento. teorema 2.4 con $L = 2$: $(I + J_2)(I + J_1) = I + J_1 + J_2 + J_2 J_1$.

Desarrollo.

1. $I + J_2 = \begin{pmatrix} 0{,}9 & 0{,}3 \\ 0 & 1{,}2 \end{pmatrix}$ e $I + J_1 = \begin{pmatrix} 1{,}2 & 0 \\ 0{,}1 & 0{,}9 \end{pmatrix}$.
2. Producto: fila 1, $(0{,}9 \cdot 1{,}2 + 0{,}3 \cdot 0{,}1;\ 0{,}3 \cdot 0{,}9) = (1{,}11;\ 0{,}27)$; fila 2, $(1{,}2 \cdot 0{,}1;\ 1{,}2 \cdot 0{,}9) = (0{,}12;\ 1{,}08)$.
3. $J_2 J_1$: fila 1, $(-0{,}1 \cdot 0{,}2 + 0{,}3 \cdot 0{,}1;\ 0{,}3 \cdot (-0{,}1)) = (0{,}01;\ -0{,}03)$; fila 2, $(0{,}2 \cdot 0{,}1;\ 0{,}2 \cdot (-0{,}1)) = (0{,}02;\ -0{,}02)$.
4. $I + J_1 + J_2 = \begin{pmatrix} 1{,}1 & 0{,}3 \\ 0{,}1 & 1{,}1 \end{pmatrix}$; sumando $J_2 J_1$ se obtiene $\begin{pmatrix} 1{,}11 & 0{,}27 \\ 0{,}12 & 1{,}08 \end{pmatrix}$.

Resultado. $(I + J_2)(I + J_1) = \begin{pmatrix} 1{,}11 & 0{,}27 \\ 0{,}12 & 1{,}08 \end{pmatrix}$**, cercano a I; la red plana daría** $J_2 J_1 = \begin{pmatrix} 0{,}01 & -0{,}03 \\ 0{,}02 & -0{,}02 \end{pmatrix}$, con valores singulares 0,041 y 0,0097.

Comprobación. los valores singulares de la matriz residual son 1,293 y 0,902, mayores que $(1-a)^2 = 0{,}397$ con $a = \max(\|J_1\|_2, \|J_2\|_2) = \max(0{,}229;\, 0{,}370) = 0{,}370$, como exige la proposición 2.7.

Solución del ejercicio 2.4

Datos. $N = 48$; $\mathrm{Var}(h_0) = 1$; $v = 0{,}5$; escalado $1/\sqrt{N}$.

Planteamiento. proposición 2.8: $\mathrm{Var}(h_N) = \mathrm{Var}(h_0) + Nv$. Multiplicar unos pesos por $1/\sqrt{N}$ multiplica la varianza de su salida por $1/N$.

Desarrollo.

1. (a) $\mathrm{Var}(h_N) = 1 + 48 \cdot 0{,}5 = 1 + 24 = 25$; desviación típica $\sqrt{25} = 5$.
2. (b) Cada aportación tiene varianza $v/N = 0{,}5/48 = 0{,}01042$.
3. $\mathrm{Var}(h_N) = 1 + 48 \cdot 0{,}01042 = 1 + 0{,}5 = 1{,}5$; desviación típica $\sqrt{1{,}5} = 1{,}225$.

Resultado. **(a) varianza 25, desviación típica 5; (b) varianza** 1,5**, desviación típica** 1,22**.**

Comprobación. en (b) la varianza total añadida es $v = 0{,}5$ con independencia de N: el escalado hace que la profundidad no cambie la escala del flujo.

Solución del ejercicio 2.5

Datos. $h_\ell = h_{\ell-1} + F_\ell(h_{\ell-1})$; caso escalar con $|J_\ell| \le a < 1$.

Planteamiento. desarrollo del producto de jacobianos y cota de cada factor.

Desarrollo.

1. (a) Cada bloque tiene jacobiano $I + J_\ell$; por la regla de la cadena, $\partial h_L/\partial h_0 = (I + J_L)\cdots(I + J_1)$.
2. Al desarrollar, cada término elige I o J_ℓ en cada factor; el que elige siempre I vale I. El desarrollo contiene, por tanto, el término I.
3. (b) $|J_\ell| \le a$ implica $J_\ell \ge -a$, luego $1 + J_\ell \ge 1 - a > 0$.
4. Todos los factores son positivos y mayores o iguales que $1 - a$; multiplicando las L desigualdades entre números positivos, $\prod_\ell (1 + J_\ell) \ge (1-a)^L > 0$. ■
5. (c) Red plana con $J_1 = 0$ (unidad ReLU apagada) y $J_2 = 3$: $J_2 J_1 = 0$. La red residual con las mismas ramas da $(1+3)(1+0) = 4$.

Resultado. **el desarrollo contiene I; con $|J_\ell| \le a < 1$ el factor escalar es al menos** $(1-a)^L > 0$; en una red plana basta una capa con jacobiano nulo para anularlo.

Comprobación. con $a = 0{,}01$ y $L = 24$, la cota es $0{,}99^{24} = 0{,}786$, el valor del ejercicio 2.2(b).

A.3 Tema 3

Solución del ejercicio 3.1

Datos. (a) $h + \mathrm{Attn}(\mathrm{LN}(h))$; (b) $\mathrm{LN}(h + \mathrm{MLP}(h))$.

Planteamiento. definición 3.9: pre-norm normaliza dentro de la rama; post-norm, después de la suma.

Desarrollo.

1. (a) La normalización está dentro de la rama: **pre-norm**. Su jacobiano es $I + J_{\text{Attn}}N$, con el término I.
2. (b) La normalización envuelve la suma: **post-norm**. Su jacobiano es $N(I + J_{\text{MLP}})$; la identidad queda multiplicada por N.
3. Con L bloques post-norm, el camino directo atraviesa L jacobianos de normalización y deja de ser la identidad (proposición 3.10).

Resultado. **(a) pre-norm; (b) post-norm. Solo (a) conserva la identidad en el camino residual.**

Comprobación. es el patrón de la figura 3.1.

Solución del ejercicio 3.2

Planteamiento. identificar qué operaciones quedan en el camino principal, fuera de la rama.

Desarrollo.

1. Ordenación original: $h_\ell = \text{ReLU}\big(h_{\ell-1} + F(h_{\ell-1})\big)$. La ReLU posterior a la suma está en el camino principal; su jacobiano $\text{diag}(\mathbf{1}[\cdot > 0])$ multiplica la identidad y puede anular componentes.
2. Ordenación de preactivación: $h_\ell = h_{\ell-1} + F(h_{\ell-1})$ con $F = \text{conv} \circ \text{ReLU} \circ \text{BN}$. Tras la suma no hay nada: el camino principal es exactamente la identidad y se aplica el teorema 2.4.
3. Pre-norm hace lo mismo con la normalización de un Transformer: la mueve dentro de la rama y deja el flujo residual libre.

Resultado. **la preactivación saca del camino principal la normalización y la no linealidad, con lo que el jacobiano de cada bloque es $I + J_\ell$; es la misma idea que pre-norm.**

Solución del ejercicio 3.3

Datos. $x = (1, 2, 3, 6)$, $d = 4$, $\gamma = g = \mathbf{1}$, $\beta = 0$, $\epsilon = 0$.

Planteamiento. definiciones 3.2 y 3.3.

Desarrollo.

1. Media: $\mu = (1 + 2 + 3 + 6)/4 = 12/4 = 3$.
2. Desviaciones: $x - \mu = (-2, -1, 0, 3)$; cuadrados $(4, 1, 0, 9)$; $\sigma^2 = 14/4 = 3{,}5$; $\sigma = 1{,}871$.
3. $\text{LN}(x) = (-2/1{,}871;\ -1/1{,}871;\ 0;\ 3/1{,}871) = (-1{,}069;\ -0{,}5345;\ 0;\ 1{,}604)$.
4. Cuadrados de x: $(1, 4, 9, 36)$; media $50/4 = 12{,}5$; $\text{RMS}(x) = \sqrt{12{,}5} = 3{,}536$.
5. $\text{RMSNorm}(x) = (0{,}2828;\ 0{,}5657;\ 0{,}8485;\ 1{,}697)$.

Resultado. $\text{LN}(x) \approx (-1{,}069;\ -0{,}5345;\ 0;\ 1{,}604)$**;**
$\text{RMSNorm}(x) \approx (0{,}2828;\ 0{,}5657;\ 0{,}8485;\ 1{,}697)$**.**

Comprobación. la salida de LayerNorm suma $-1{,}069-0{,}5345+0+1{,}604 \approx 0$ y la media de sus cuadrados es $(1{,}143+0{,}2857+0+2{,}571)/4 = 1{,}000$. La media de los cuadrados de RMSNorm es $(0{,}08+0{,}32+0{,}72+2{,}88)/4 = 1$.

Solución del ejercicio 3.4

Datos. $x = (1,2,3,6)$; $2x + 10\mathbf{1} = (12,14,16,22)$; $x + 3\mathbf{1} = (4,5,6,9)$.

Planteamiento. recalcular los estadísticos y comparar con el ejercicio 3.3.

Desarrollo.

1. (a) $\mu = 64/4 = 16$; desviaciones $(-4,-2,0,6)$; cuadrados $(16,4,0,36)$; $\sigma^2 = 56/4 = 14$; $\sigma = 3{,}742$.
2. $\mathrm{LN}(2x+10\mathbf{1}) = (-4,-2,0,6)/3{,}742 = (-1{,}069;\ -0{,}5345;\ 0;\ 1{,}604)$, igual que $\mathrm{LN}(x)$.
3. (b) Cuadrados de $x+3\mathbf{1}$: $(16,25,36,81)$; media $158/4 = 39{,}5$; RMS $= 6{,}285$.
4. $\mathrm{RMSNorm}(x+3\mathbf{1}) = (0{,}6364;\ 0{,}7956;\ 0{,}9547;\ 1{,}432)$, distinto de $\mathrm{RMSNorm}(x) = (0{,}2828;\ 0{,}5657;\ 0{,}8485;\ 1{,}697)$.

Resultado. **(a) coinciden: invariancia de LayerNorm frente a escala y traslación (proposición 3.6(i)); (b) no coinciden: RMSNorm no es invariante frente a traslaciones (proposición 3.6(ii)).**

Comprobación. en (a) la desviación típica se multiplica por $\alpha = 2$: $3{,}742 = 2 \cdot 1{,}871$.

Solución del ejercicio 3.5

Datos. $x = (3,4)$, $d = 2$, $g = \mathbf{1}$, $\epsilon = 0$.

Planteamiento. teorema 3.7: $J = \frac{1}{r}\big(I - xx^\top/\|x\|^2\big)$.

Desarrollo.

1. $\|x\|^2 = 9+16 = 25$; $r = \sqrt{25/2} = 3{,}536$.
2. $xx^\top/25 = \left(\begin{smallmatrix} 9 & 12 \\ 12 & 16 \end{smallmatrix}\right)/25 = \left(\begin{smallmatrix} 0{,}36 & 0{,}48 \\ 0{,}48 & 0{,}64 \end{smallmatrix}\right)$; $I - xx^\top/25 = \left(\begin{smallmatrix} 0{,}64 & -0{,}48 \\ -0{,}48 & 0{,}36 \end{smallmatrix}\right)$.
3. $J = \frac{1}{3{,}536}\left(\begin{smallmatrix} 0{,}64 & -0{,}48 \\ -0{,}48 & 0{,}36 \end{smallmatrix}\right) = \left(\begin{smallmatrix} 0{,}1810 & -0{,}1358 \\ -0{,}1358 & 0{,}1018 \end{smallmatrix}\right)$.
4. $Jx = \frac{1}{r}(0{,}64\cdot 3 - 0{,}48\cdot 4;\ -0{,}48\cdot 3 + 0{,}36\cdot 4) = \frac{1}{r}(1{,}92 - 1{,}92;\ -1{,}44 + 1{,}44) = (0,0)$.

Resultado. $J \approx \left(\begin{smallmatrix} 0{,}1810 & -0{,}1358 \\ -0{,}1358 & 0{,}1018 \end{smallmatrix}\right)$**, simétrica y con** $Jx = 0$**.**

Comprobación. la diferenciación automática de PyTorch da la misma matriz. Además, $(-4,3) \perp x$ y $J(-4,3) = (-4,3)/r$: en la dirección ortogonal, J actúa como $1/r$ por la identidad.

Solución del ejercicio 3.6

Datos. $y = \mathrm{RMSNorm}(x)$, $\epsilon = 0$ salvo en (d); $\mathcal{L} = \mathcal{L}(y)$.

Planteamiento. teorema 3.7 y regla de la cadena.

Desarrollo.

1. (a) $\mathrm{RMS}(\alpha x) = \sqrt{\alpha^2 \frac{1}{d}\sum x_i^2} = \alpha\,\mathrm{RMS}(x)$ para $\alpha > 0$, luego $g_i \alpha x_i/(\alpha\,\mathrm{RMS}(x)) = \mathrm{RMSNorm}(x)_i$.

2. Derivando $\phi(\alpha) = \text{RMSNorm}(\alpha x)$, constante, en $\alpha = 1$: $0 = \phi'(1) = J\,x$.
3. (b) $\nabla_x \mathcal{L} = J^\top \nabla_y \mathcal{L}$ y $x^\top J^\top \nabla_y \mathcal{L} = (Jx)^\top \nabla_y \mathcal{L} = 0$.
4. (c) $\frac{d}{dt}\|x\|^2 = 2x^\top \dot{x} = -2x^\top \nabla_x \mathcal{L} = 0$: la norma es constante.
5. Con un paso discreto, $\|x - \eta \nabla_x \mathcal{L}\|^2 = \|x\|^2 - 2\eta\, x^\top \nabla_x \mathcal{L} + \eta^2 \|\nabla_x \mathcal{L}\|^2 = \|x\|^2 + \eta^2 \|\nabla_x \mathcal{L}\|^2$, mayor que $\|x\|^2$ si el gradiente no es nulo.
6. (d) Con $\epsilon > 0$, $r^2 = s + \epsilon$ con $s = \|x\|^2/d$. Repitiendo el cálculo, $(Jx)_i = \frac{g_i x_i}{r}\left(1 - \frac{s}{r^2}\right) = \frac{g_i x_i\, \epsilon}{r^3}$, que no es nulo pero es despreciable si $s \gg \epsilon$.

Resultado. $Jx = 0$ **y** $x^\top \nabla_x \mathcal{L} = 0$**; la norma se conserva en el flujo continuo y crece en cada paso discreto. Con** $\epsilon > 0$ **la invariancia es solo aproximada.**

Comprobación. en el ejercicio 3.5, $x^\top J^\top v = (Jx)^\top v = 0$ para todo v, porque $Jx = 0$ y J es simétrica.

A.4 Tema 4

Solución del ejercicio 4.1

Planteamiento. el dropout invertido conserva la esperanza, pero cada máscara es aleatoria (proposición 4.2).

Desarrollo.

1. Falta poner el modelo en modo evaluación (`model.eval()` en PyTorch), que desactiva el dropout.
2. Con dropout activo, cada pasada sortea una máscara distinta y la salida cambia.
3. La media no lo delata: $\mathbb{E}[\tilde{h}_i] = (1-p)\, h_i/(1-p) + p \cdot 0 = h_i$.
4. Lo que cambia es la varianza, $\frac{p}{1-p} h_i^2$: cada ejecución concreta tiene ruido.

Resultado. falta `model.eval()`; **el dropout invertido es insesgado, por lo que la media coincide, pero cada ejecución tiene ruido.**

Comprobación. con $p = 0{,}3$ y $h_i = 5$, $0{,}7 \cdot 7{,}143 + 0{,}3 \cdot 0 = 5{,}000$ (ejercicio 4.3).

Solución del ejercicio 4.2

Datos. $P(d_i = 0) = p$; $p = 0{,}2$; $h = (1, 3, 0, 2)$; $d = (1, 0, 1, 1)$.

Planteamiento. $\mathbb{E}[d_i] = 1 - p$ y linealidad de la esperanza.

Desarrollo.

1. (a) $\mathbb{E}[h_{\text{drop},i}] = \gamma h_i \mathbb{E}[d_i] = \gamma h_i (1-p)$. Igualando a h_i para $h_i \neq 0$: $\gamma = 1/(1-p)$.
2. (b) $\gamma = 1/0{,}8 = 1{,}25$; $d \odot h = (1, 0, 0, 2)$; $h_{\text{drop}} = (1{,}25;\ 0;\ 0;\ 2{,}5)$.
3. (c) Al entrenar, la máscara impide que la red dependa de unidades concretas y actúa como regularizador. Al evaluar se quiere una predicción determinista, la del modelo medio; gracias a γ, usar h sin máscara da la esperanza correcta.

Resultado. $\gamma = 1/(1-p) = 1{,}25$**;** $h_{\text{drop}} = (1{,}25;\ 0;\ 0;\ 2{,}5)$**.** Es el dropout invertido de la definición 4.1.

Comprobación. $\mathbb{E}[h_{\text{drop},2}] = 0{,}8 \cdot 1{,}25 \cdot 3 + 0{,}2 \cdot 0 = 3 = h_2$.

Solución del ejercicio 4.3

Datos. $p = 0{,}3$; $h = (5, 5, 5, 5)$; $m = (1, 0, 1, 1)$.

Planteamiento. $\tilde{h}_i = m_i h_i/(1-p)$.

Desarrollo.

1. $1 - p = 0{,}7$; factor $1/0{,}7 = 1{,}429$.
2. $m \odot h = (5, 0, 5, 5)$; $\tilde{h} = (7{,}143;\ 0;\ 7{,}143;\ 7{,}143)$.
3. $\mathbb{E}[\tilde{h}_i] = 0{,}7 \cdot (5/0{,}7) + 0{,}3 \cdot 0 = 5 = h_i$.
4. Media de la salida para esta máscara: $3 \cdot 7{,}143/4 = 5{,}357$.

Resultado. $\tilde{h} \approx (7{,}143;\ 0;\ 7{,}143;\ 7{,}143)$**, con** $\mathbb{E}[\tilde{h}_i] = 5$**; la media de esta realización es** $5{,}357$**.**

Comprobación. con una fracción $3/4$ de supervivientes, la media es $5 \cdot (3/4)/0{,}7 = 5{,}357$; promediando sobre muchas máscaras la fracción tiende a 0,7 y la media a 5.

Solución del ejercicio 4.4

Datos. $g = (6, 8)$; $c = 5$ y $c = 20$.

Planteamiento. $\hat{g} = g \min(1, c/\|g\|)$ (definición 4.6).

Desarrollo.

1. $\|g\| = \sqrt{36 + 64} = \sqrt{100} = 10$.
2. $c = 5$: $\alpha = \min(1;\ 0{,}5) = 0{,}5$; $\hat{g} = (3, 4)$.
3. $c = 20$: $\alpha = \min(1;\ 2) = 1$; $\hat{g} = (6, 8)$.

Resultado. $c = 5$**:** $\hat{g} = (3, 4)$**;** $c = 20$**:** $\hat{g} = (6, 8)$**, sin recorte.**

Comprobación. $\|(3, 4)\| = 5 = \min(10, 5)$ y la dirección $(0{,}6;\ 0{,}8)$ se conserva, como afirma la proposición 4.7(i).

Solución del ejercicio 4.5

Datos. $g^{(1)} = (1, 2)$, $g^{(2)} = (2)$, $c = 1$.

Planteamiento. la norma global es la del vector concatenado; el mismo factor se aplica a todos los tensores.

Desarrollo.

1. (a) $g = (1, 2, 2)$, $\|g\| = \sqrt{1 + 4 + 4} = 3$; $\alpha = 1/3$.
2. $\hat{g}^{(1)} = (0{,}3333;\ 0{,}6667)$ y $\hat{g}^{(2)} = (0{,}6667)$.
3. (b) $\|g^{(1)}\| = \sqrt{5} = 2{,}236$, luego $\hat{g}^{(1)} = (0{,}4472;\ 0{,}8944)$; $\|g^{(2)}\| = 2$, luego $\hat{g}^{(2)} = (1)$.
4. El vector $(0{,}4472;\ 0{,}8944;\ 1)$ no es proporcional a $(1, 2, 2)$: el cociente entre la tercera y la primera componente es 2,236 en lugar de 2.

Resultado. **norma global:** $\hat{g}^{(1)} = (1/3,\ 2/3)$**,** $\hat{g}^{(2)} = (2/3)$**. Recortar por separado cambia la dirección del gradiente total.**

Comprobación. $\|(1/3, 2/3, 2/3)\| = \sqrt{1/9 + 4/9 + 4/9} = 1 = c$.

Solución del ejercicio 4.6

Datos. $g^{(1)} = (2, -1)$, $g^{(2)} = (2)$, $M = 1$, $\epsilon = 10^{-6}$.

Planteamiento. factor $M/(\|g\| + \epsilon)$ si $\|g\| \geq M$.

Desarrollo.

1. (a) $\|g\| = \sqrt{4 + 1 + 4} = 3 \geq 1$: hay que recortar.
2. Factor: $1/(3 + 10^{-6}) = 0{,}3333332$.
3. $\hat{g}^{(1)} = (0{,}6667;\ -0{,}3333)$ y $\hat{g}^{(2)} = (0{,}6667)$.
4. (b) Norma resultante: $3/(3 + 10^{-6}) = 0{,}9999997$.
5. El ϵ evita la división por cero cuando $g = 0$ y deja la norma ligeramente por debajo de M.

Resultado. **norma global 3 y factor** $\approx 1/3$**;** $\hat{g}^{(1)} \approx (0{,}6667;\ -0{,}3333)$**,** $\hat{g}^{(2)} \approx (0{,}6667)$**; norma final** $0{,}9999997$**.**

Comprobación. `torch.nn.utils.clip_grad_norm_` devuelve la norma original, 3, y deja los mismos gradientes.

Solución del ejercicio 4.7

Datos. $w = (1, 2, -1)$, $h = (2, 1, 2)$, $p = 0{,}5$; $m_i \sim$ Bernoulli$(0{,}5)$ independientes.

Planteamiento. proposición 4.2 y corolario 4.3; enumeración de las 2^3 máscaras equiprobables.

Desarrollo.

1. (a) Las demostraciones son las del texto: $\mathbb{E}[m_i] = \mathbb{E}[m_i^2] = 1 - p$ da $\mathbb{E}[\tilde{h}_i] = h_i$ y $\mathbb{E}[\tilde{h}_i^2] = h_i^2/(1-p)$, de donde $\mathrm{Var}(\tilde{h}_i) = h_i^2 p/(1-p)$; la independencia de las máscaras da la suma de varianzas.
2. (b) $\mathbb{E}[y] = w^\top h = 1 \cdot 2 + 2 \cdot 1 + (-1) \cdot 2 = 2$.
3. $\mathrm{Var}(y) = \frac{0{,}5}{0{,}5}(1 \cdot 4 + 4 \cdot 1 + 1 \cdot 4) = 12$.
4. Enumeración: $w_i h_i/(1-p) = (4, 4, -4)$, luego $y = 4(m_1 + m_2 - m_3)$. Las ocho máscaras dan 0, 4, 4, -4, 8, 0, 0, 4, cada una con probabilidad $1/8$.
5. Media: $16/8 = 2$. Segundo momento: $(0+16+16+16+64+0+0+16)/8 = 128/8 = 16$. Varianza: $16 - 2^2 = 12$.
6. (c) Sin dropout, la activación h_i es fija dada la entrada; con dropout, su segundo momento pasa a $h_i^2/(1-p)$ y su varianza a $\frac{p}{1-p}h_i^2$, que tiende a infinito cuando $p \to 1$ porque las pocas unidades supervivientes se amplifican por $1/(1-p)$.

Resultado. $\mathbb{E}[y] = 2$ **y** $\mathrm{Var}(y) = 12$**, confirmados por enumeración; la varianza que añade el dropout es** $\frac{p}{1-p}h_i^2$ **por componente.**

Comprobación. la desviación típica es $\sqrt{12} = 3{,}464$, mayor que la media: con $p = 0{,}5$ el ruido es grande.

A.5 Tema 5

Solución del ejercicio 5.1

Datos. 70 000 y 1,003; fp16 ($x_{\max} = 65\,504$, $\varepsilon = 2^{-10}$), bf16 ($x_{\max} = 3{,}39 \cdot 10^{38}$, $\varepsilon = 2^{-7}$), fp32.

Planteamiento. desbordamiento si el valor supera $x_{\max}$; pérdida de precisión según la separación cerca de 1, que es ε.

Desarrollo.

1. $70\,000 > 65\,504$: en fp16 se convierte en `inf`. En bf16 cabe, redondeado a 70 144 (separación $2^{16} \cdot 2^{-7} = 512$). En fp32 es exacto.
2. Cerca de 1, bf16 separa los números $2^{-7} = 0{,}0078$; los vecinos de 1,003 son 1 y 1,0078, y el más cercano es 1. Error: 0,003.
3. fp16 los separa $2^{-10} = 0{,}000977$; 1,003 se redondea a $1 + 3 \cdot 2^{-10} = 1{,}00293$. Error: $7 \cdot 10^{-5}$.

Resultado. 70 000 **es arriesgado en fp16 (da `inf`);** 1,003 **pierde más precisión en bf16 (se almacena como 1).**

Comprobación. en PyTorch, 1,003 en bf16 da 1,0 y en fp16 da 1,0029.

Solución del ejercicio 5.2

Datos. fp16: $e = 5$, $m = 10$; bf16: $e = 8$, $m = 7$.

Planteamiento. proposición 5.2: $b = 2^{e-1} - 1$, $x_{\max} = (2 - 2^{-m})2^b$, $x_{\min} = 2^{1-b}$, $\varepsilon = 2^{-m}$.

Desarrollo.

1. fp16: $b = 2^4 - 1 = 15$.
2. $x_{\max} = (2 - 2^{-10})2^{15} = 2^{16} - 2^5 = 65\,536 - 32 = 65\,504$.
3. $x_{\min} = 2^{-14} = 1/16\,384 = 6{,}104 \cdot 10^{-5}$; $\varepsilon = 2^{-10} = 9{,}766 \cdot 10^{-4}$.
4. bf16: $b = 2^7 - 1 = 127$.
5. $x_{\max} = (2 - 2^{-7})2^{127} = 1{,}9922 \cdot 1{,}7014 \cdot 10^{38} = 3{,}390 \cdot 10^{38}$.
6. $x_{\min} = 2^{-126} = 1{,}175 \cdot 10^{-38}$; $\varepsilon = 2^{-7} = 0{,}0078125$.

Resultado. **fp16:** 65 504**,** $6{,}10 \cdot 10^{-5}$**,** $9{,}77 \cdot 10^{-4}$**. bf16:** $3{,}39 \cdot 10^{38}$**,** $1{,}18 \cdot 10^{-38}$**,** 0,0078125**.**

Comprobación. bf16 tiene casi el rango de fp32 ($3{,}40 \cdot 10^{38}$ y $1{,}18 \cdot 10^{-38}$) porque comparten los 8 bits de exponente.

Solución del ejercicio 5.3

Datos. E5M2: $e = 5$, $m = 2$; E4M3: $e = 4$, $m = 3$.

Planteamiento. proposición 5.2 y observación 5.4.

Desarrollo.

1. (a) E5M2: $b = 15$; $x_{\max} = (2 - 2^{-2})2^{15} = 1{,}75 \cdot 32\,768 = 57\,344$; $x_{\min} = 2^{-14} = 6{,}10 \cdot 10^{-5}$; $\varepsilon = 2^{-2} = 0{,}25$.

2. (b) E4M3 con reglas IEEE: $b = 2^3 - 1 = 7$; el mayor exponente normal sería $E = 14$, con $E - b = 7$; $x_{\max} = (2 - 2^{-3})2^7 = 1{,}875 \cdot 128 = 240$; $x_{\min} = 2^{-6} = 0{,}015625$; $\varepsilon = 2^{-3} = 0{,}125$.

3. E4M3 real: sin infinitos y con un único patrón `NaN` (todos los bits de exponente y fracción a 1), el exponente $E = 15$ es normal salvo en ese patrón. La mayor mantisa permitida con $E = 15$ es $(1{,}110)_2 = 1{,}75$, y $x_{\max} = 1{,}75 \cdot 2^8 = 448$.

4. (c) Los gradientes abarcan muchos órdenes de magnitud y requieren rango: E5M2. Pesos y activaciones requieren más precisión relativa: E4M3.

Resultado. **E5M2:** $57\,344$**,** $6{,}10 \cdot 10^{-5}$**,** $0{,}25$**. E4M3: con reglas IEEE serían** 240**,** $0{,}0156$**,** $0{,}125$**; el máximo real es 448.**

Comprobación. `torch.finfo` da 448 y 57 344 como máximos de `float8_e4m3fn` y `float8_e5m2`.

Solución del ejercicio 5.4

Datos. bf16 en $[512, 1024)$: separación 4; fp16 en $[512, 1024)$: separación 0,5.

Planteamiento. el resultado exacto se redondea al representable más cercano; en caso de empate, al de mantisa par (definición 5.5).

Desarrollo.

1. (a) $1000 + 1 = 1001$; vecinos representables en bf16: 1000 y 1004, a distancias 1 y 3. Resultado: 1000.

2. $1000 + 4 = 1004 = 4 \cdot 251$, representable: resultado exacto.

3. (b) En fp16, $1001 = 0{,}5 \cdot 2002$ es representable: resultado exacto.

4. (c) Por el lema 5.7, se pierde todo sumando $0 \le y < 2$, la mitad de la separación. Con $y = 2$ hay empate entre $1000 = 4 \cdot 250$ y $1004 = 4 \cdot 251$, y gana el de mantisa par, 1000; con $y > 2$ el resultado es 1004.

Resultado. **bf16:** $1000 + 1 = 1000$ **y** $1000 + 4 = 1004$**; fp16:** $1000 + 1 = 1001$**. En bf16 se pierde cualquier sumando de hasta 2.**

Comprobación. PyTorch da 1000 para $1000 + 2$ y 1004 para $1000 + 2{,}5$ en bf16.

Solución del ejercicio 5.5

Datos. $x = (300, 400)$ en fp16 ($x_{\max} = 65\,504$); $g = \mathbf{1}$; $\epsilon = 10^{-5}$; $d = 2$.

Planteamiento. $\mathrm{RMSNorm}(x)_i = x_i / \sqrt{\frac{1}{d}\sum x_j^2 + \epsilon}$; el cuadrado es lo que desborda.

Desarrollo.

1. (a) En fp16: $300^2 = 90\,000 > 65\,504 \to$ `inf`; $400^2 = 160\,000 \to$ `inf`.

2. Media `inf`; RMS $= \sqrt{\texttt{inf} + 10^{-5}} =$ `inf`; salida $(300/\texttt{inf},\ 400/\texttt{inf}) = (0, 0)$. La señal se destruye sin ningún error.

3. (b) En fp32: $x^2 = (90\,000;\ 160\,000)$; media $125\,000$; RMS $= \sqrt{125\,000{,}00001} = 353{,}6$.

4. Salida: $(300/353{,}6;\ 400/353{,}6) = (0{,}8485;\ 1{,}131)$.

5. En fp16, con separación 2^{-11} en $[0{,}5; 1)$ y 2^{-10} en $[1, 2)$: $(0{,}8486;\ 1{,}132)$.

Resultado. **todo en fp16:** $(0, 0)$**; en fp32 convertido a fp16:** $(0{,}8486;\ 1{,}132)$**.**

Comprobación. $300/\sqrt{125\,000} = 3/(5\sqrt{2}) = 0{,}8485$ y $4/(5\sqrt{2}) = 1{,}131$; la media de sus cuadrados es $(0{,}72 + 1{,}28)/2 = 1$.

Solución del ejercicio 5.6

Datos. formato con m bits de mantisa; fp16, $m = 10$.

Planteamiento. proposición 5.2(iii): en $[2^k, 2^{k+1})$ la separación es 2^{k-m}.

Desarrollo.

1. (a) Sea $2^k \le x < 2^{k+1}$. Los representables vecinos de x distan 2^{k-m} entre sí, luego $|\mathrm{fl}(x) - x| \le 2^{k-m-1}$. Dividiendo por $x \ge 2^k$: $|\delta| \le 2^{-m-1} = u$.
2. (b) El representable siguiente a x es $x+2^{k-m}$. Si $0 \le y < 2^{k-m-1}$, entonces $|(x+y)-x| = y < 2^{k-m} - y = |(x + 2^{k-m}) - (x + y)|$, y $\mathrm{fl}(x + y) = x$.
3. (c) Sea $w \in [2^k, 2^{k+1})$. Si $w/|\Delta w| > 4096 = 2^{12}$, entonces $|\Delta w| < w/2^{12} < 2^{k+1}/2^{12} = 2^{k-11}$, la mitad de la separación en fp16, y por (b) la actualización se pierde.
4. Ejemplo: $w = 1$ $(k = 0)$, semiseparación $2^{-11} = 4{,}883 \cdot 10^{-4}$. Con $\Delta w = 4{,}8 \cdot 10^{-4}$, $w/\Delta w = 2083 > 2048$ y $\mathrm{fl}(1 + 0{,}00048) = 1$.

Resultado. $|\delta| \le u$**; un sumando menor que la semiseparación se absorbe; en fp16 una actualización se pierde siempre si** $w/|\Delta w| > 4096$**, y puede perderse desde** 2048**.** ■

Comprobación. en PyTorch, $1 + 0{,}00048$ en fp16 da 1, mientras que $1 + 0{,}0005$ da 1,000977, porque 0,0005 supera la semiseparación.

Solución del ejercicio 5.7

Datos. mil sumas de 0,01; fp16 con $m = 10$; en $[2^k, 2^{k+1})$ la separación es 2^{k-10}.

Planteamiento. cada suma se redondea al múltiplo más próximo de la separación del intervalo en que está s; el error por suma depende de cuántas separaciones mide el sumando.

Desarrollo.

1. (a) $0{,}01 \in [2^{-7}, 2^{-6})$, con separación $2^{-7-10} = 2^{-17} = 7{,}629 \cdot 10^{-6}$. Como $0{,}01/2^{-17} = 1310{,}72$, se redondea a 1311 separaciones: $1311 \cdot 2^{-17} = 0{,}0100021$.
2. (b) $s \in [4, 8)$: separación $2^{-8} = 0{,}003906$; el sumando mide 2,561 separaciones y se redondea a 3, de modo que cada suma aumenta s en $3 \cdot 2^{-8} = 0{,}01172$.
3. $s \in [8, 16)$: separación $2^{-7} = 0{,}0078125$; el sumando mide 1,280 separaciones y se redondea a 1: cada suma aumenta s en 0,0078125.
4. (c) (i) En fp32 la separación cerca de 10 es $2^{3-23} = 9{,}5 \cdot 10^{-7}$ y el error acumulado es pequeño: 10,0001.
5. (ii) En fp16 el error por suma cambia de signo según el intervalo. En $[0{,}5; 4)$ cada suma aumenta s en 0,009766, por defecto; en $[4, 8)$, en 0,01172, por exceso; en $[8, 10)$, en 0,0078125, por defecto. La simulación paso a paso da 359 sumas en $[0{,}5; 4)$ con error total $-0{,}0830$; 341 en $[4, 8)$ con $+0{,}5815$; 250 en $[8, 10)$ con $-0{,}5474$; y 50 por debajo de 0,5 con $-0{,}0001$.
6. El neto es $-0{,}0490$, y $1000 \cdot 0{,}0100021 - 0{,}0490 = 10{,}0021 - 0{,}0490 = 9{,}953$.

7. (iii) y (iv): el acumulador fp32 apenas pierde nada, pero el sumando ya es el 0,01 de fp16: $1000 \cdot 0{,}0100021 = 10{,}0021$. En (iii) la conversión a fp32 se hace implícitamente antes de sumar; en (iv), explícitamente.

Resultado. **(a)** $0{,}01 \approx 1311 \cdot 2^{-17}$**; (b)** $+0{,}01172$ **por suma en** $[4, 8)$ **y** $+0{,}0078125$ **en** $[8, 16)$**; (c) el error de (ii) procede del acumulador fp16; el de (iii) y (iv), solo del redondeo del sumando.**

Comprobación. $359 + 341 + 250 + 50 = 1000$ sumas. Es el argumento de la acumulación en fp32 de la precisión mixta y del teorema 5.10.

A.6 Tema 6

Solución del ejercicio 6.1

Planteamiento. cuadro 6.1 y figura 6.1.

Desarrollo.

(a) Salto brusco a `inf` en fp16 sin crecimiento previo: desbordamiento de alguna activación o gradiente por encima de 65 504. Remedio: bf16, o escalado dinámico de la pérdida que omita el paso y reduzca S.

(b) Crecimiento de varios órdenes y después `NaN`: gradientes que explotan. Remedio: bajar la tasa de aprendizaje, alargar el calentamiento y recortar el gradiente.

(c) `torch.log(torch.softmax(z))`: alguna probabilidad se anula por subdesbordamiento y $\ln 0 = -\infty$; en la retropropagación aparecen operaciones como $0 \cdot \infty$ que dan `NaN`. Remedio: `log_softmax` o la entropía cruzada aplicada a los logits (proposición 6.6).

Resultado. **(a) desbordamiento en fp16; (b) gradientes que explotan; (c) logaritmo de una probabilidad nula.**

Solución del ejercicio 6.2

Datos. $z = (1000, 999, 998)$.

Planteamiento. proposición 6.4 con $c = 1000$.

Desarrollo.

1. $z - c\mathbf{1} = (0, -1, -2)$.
2. $e^0 = 1$; $e^{-1} = 0{,}3679$; $e^{-2} = 0{,}1353$; $S = 1{,}503$.
3. $\ln 1{,}503 = 0{,}4076$; $\mathrm{LSE}(z) = 1000{,}408$.
4. $\mathrm{softmax}(z) = (1/1{,}503;\ 0{,}3679/1{,}503;\ 0{,}1353/1{,}503) = (0{,}665;\ 0{,}245;\ 0{,}090)$.

Resultado. $\mathrm{LSE}(z) \approx 1000{,}408$ **y** $\mathrm{softmax}(z) \approx (0{,}665;\ 0{,}245;\ 0{,}090)$**.**

Comprobación. las probabilidades suman 1,000 y $1000 \le 1000{,}408 \le 1000 + \ln 3 = 1001{,}099$.

Solución del ejercicio 6.3

Datos. $v = (100, 101, 103)$ en fp32; $\ln x_{\max} = 88{,}72$.

Planteamiento. proposición 6.1 y proposición 6.3(i).

Desarrollo.

1. (a) $100 > 88{,}72$, luego $e^{100} = 2{,}7 \cdot 10^{43}$ desborda a `inf`, igual que e^{101} y e^{103}. Cada componente es `inf`/`inf` = `NaN`.
2. (b) $\operatorname{softmax}(v - c\mathbf{1})_i = \frac{e^{v_i} e^{-c}}{e^{-c} \sum_j e^{v_j}} = \operatorname{softmax}(v)_i$, porque e^{-c} sale de la suma y se cancela. ■
3. (c) $c = 103$; $v - c\mathbf{1} = (-3, -2, 0)$; $e^{-3} = 0{,}04979$, $e^{-2} = 0{,}1353$, $e^{0} = 1$; suma 1,185.
4. $\operatorname{softmax}(v) = (0{,}04979/1{,}185;\ 0{,}1353/1{,}185;\ 1/1{,}185) = (0{,}04201;\ 0{,}1142;\ 0{,}8438)$.

Resultado. **ingenua:** (`NaN`, `NaN`, `NaN`)**; estable:** (0,0420; 0,1142; 0,8438)**.**

Comprobación. las componentes suman 1,000; `torch.softmax` en fp32 da lo mismo.

Solución del ejercicio 6.4

Datos. $x_{\max}$ y $x_{\min}$ de la cuadro 5.1.

Planteamiento. proposición 6.1: $t \in [\ln x_{\min}, \ln x_{\max}]$.

Desarrollo.

1. fp32: $\ln(3{,}403 \cdot 10^{38}) = 88{,}72$; $\ln 2^{-126} = -126 \ln 2 = -87{,}34$.
2. fp16: $\ln 65\,504 = 11{,}09$; $\ln 2^{-14} = -14 \ln 2 = -9{,}704$.
3. bf16: $\ln(3{,}390 \cdot 10^{38}) = 88{,}72$; $\ln 2^{-126} = -87{,}34$.

Resultado. **fp32 y bf16:** $t \in [-87{,}34;\ 88{,}72]$**; fp16:** $t \in [-9{,}70;\ 11{,}09]$**. fp16 es el más restrictivo: un logit de 12 ya desborda.**

Comprobación. en PyTorch, $e^{11{,}0}$ en fp16 da 59 872 y $e^{11{,}1}$ da `inf`; en fp32, $e^{88{,}7}$ es finito y $e^{88{,}8}$ es `inf`.

Solución del ejercicio 6.5

Datos. (a) $z = (2, 1, 0)$, $y = 3$; (b) $z = (0, -20)$ en fp16.

Planteamiento. proposición 6.6: $\ell = \operatorname{LSE}(z) - z_y$ y $\nabla_z \ell = \operatorname{softmax}(z) - e_y$.

Desarrollo.

1. (a) $\operatorname{LSE}(z) = 2 + \ln(1 + e^{-1} + e^{-2}) = 2 + 0{,}4076 = 2{,}408$; $\ell = 2{,}408 - 0 = 2{,}408$.
2. $\operatorname{softmax}(z) = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$; $\nabla_z \ell = (0{,}6652;\ 0{,}2447;\ -0{,}9100)$.
3. (b) Ingenua: $e^{-20} = 2{,}06 \cdot 10^{-9}$ es menor que la mitad del menor subnormal de fp16, $2^{-25} = 2{,}98 \cdot 10^{-8}$, y se redondea a 0; $\operatorname{softmax}(z)_2 = 0$ y $\ln 0 = -\infty$.
4. Estable: $\operatorname{LSE}(z) = 0 + \ln(1 + e^{-20}) = 2{,}06 \cdot 10^{-9}$, y $\ln \operatorname{softmax}(z)_2 = -20 - 2{,}06 \cdot 10^{-9} = -20{,}000$.

Resultado. **(a)** $\ell = 2{,}408$ **y** $\nabla_z \ell = (0{,}665;\ 0{,}245;\ -0{,}910)$**; (b) ingenua** $-\infty$**, estable** $-20{,}000$**.**

Comprobación. las componentes del gradiente suman 0 (salvo redondeo), porque $\operatorname{softmax}(z)$ y e_y suman 1. PyTorch da `-inf` con `log(softmax)` y -20 con `log_softmax` en fp16.

Solución del ejercicio 6.6

Datos. $z \in \mathbb{R}^n$, $c = \max_i z_i$, $S = \sum_i e^{z_i - c}$.

Planteamiento. $\operatorname{LSE}(z) = c + \ln S$ (proposición 6.4) y monotonía del logaritmo.

Desarrollo.

1. Cada $z_i - c \le 0$, luego cada término $e^{z_i - c} \in (0, 1]$.
2. Cota inferior: S contiene el término $e^0 = 1$ del máximo y los demás son positivos, así que $S \ge 1$ y $\ln S \ge 0$: $\mathrm{LSE}(z) \ge c$.
3. Cota superior: S es suma de n términos menores o iguales que 1, así que $S \le n$ y $\mathrm{LSE}(z) \le c + \ln n$.
4. Igualdad superior: $S = n$ exige que todos los términos valgan 1, es decir, $z_i = c$ para todo i; recíprocamente, si todas las componentes son iguales, $S = n$. ■
5. $z = (3, 3)$: $\mathrm{LSE} = 3 + \ln 2 = 3{,}693$, la cota superior. $z = (2, 1, 0)$: $\mathrm{LSE} = 2{,}408 \in [2;\ 2 + \ln 3] = [2;\ 3{,}099]$.

Resultado. $\max_i z_i \le \mathrm{LSE}(z) \le \max_i z_i + \ln n$**, con igualdad superior si y solo si todas las** z_i **son iguales.**

Comprobación. la cota inferior se alcanza solo en el límite: $\mathrm{LSE}(t, 0) \to t$ cuando $t \to +\infty$, porque $\ln(1 + e^{-t}) \to 0$.

Solución del ejercicio 6.7

Datos. $p = \mathrm{softmax}(z)$; $\nabla \mathrm{LSE} = p$ (proposición 6.3(iii)).

Planteamiento. derivar $p_k = e^{z_k} / \sum_i e^{z_i}$ y estudiar la forma cuadrática del hessiano.

Desarrollo.

1. (a) Por la regla del cociente, $\partial p_k / \partial z_j = p_k \delta_{jk} - p_k p_j$. Como $\partial^2 \mathrm{LSE} / \partial z_j \partial z_k = \partial p_k / \partial z_j$, el hessiano es $\mathrm{diag}(p) - pp^\top$.
2. (b) $v^\top \mathrm{diag}(p) v = \sum_i p_i v_i^2$ y $v^\top pp^\top v = (\sum_i p_i v_i)^2 = \bar{v}^2$.
3. Usando $\sum_i p_i = 1$: $\sum_i p_i (v_i - \bar{v})^2 = \sum_i p_i v_i^2 - 2\bar{v} \sum_i p_i v_i + \bar{v}^2 = \sum_i p_i v_i^2 - \bar{v}^2$, que es la forma cuadrática. Es una suma de términos no negativos.
4. Un hessiano semidefinido positivo en todo punto implica convexidad (módulo 4). La entropía cruzada $\ell = \mathrm{LSE}(z) - z_y$ es la suma de una función convexa y una lineal, luego es convexa.
5. (c) Con $v = \mathbf{1}$, $\bar{v} = 1$ y $v_i - \bar{v} = 0$ para todo i: la forma cuadrática es nula y $(\mathrm{diag}(p) - pp^\top)\mathbf{1} = p - p(p^\top \mathbf{1}) = 0$. Es coherente con la proposición 6.3(i): a lo largo de la dirección $\mathbf{1}$, LSE es lineal y su segunda derivada se anula.

Resultado. $\nabla^2 \mathrm{LSE} = \mathrm{diag}(p) - pp^\top \succeq 0$**, con 1 en su núcleo;** LSE **y la entropía cruzada son convexas en los logits.** ■

Comprobación. para $n = 2$ y $z = (0, 0)$, $p = (0{,}5;\ 0{,}5)$ y el hessiano es $\begin{pmatrix} 0{,}25 & -0{,}25 \\ -0{,}25 & 0{,}25 \end{pmatrix}$, con autovalores 0,5 y 0.

Bibliografía

[1] J. L. Ba, J. R. Kiros y G. E. Hinton. Layer normalization. Prepublicación, 2016. https://arxiv.org/abs/1607.06450

[2] C. M. Bishop y H. Bishop. *Deep Learning: Foundations and Concepts*. Springer, 2024. Cap. 7, «Gradient Descent» (§7.4 «Normalization»); cap. 9, «Regularization» (§9.5 «Residual Connections» y §9.6.1 «Dropout»). https://www.bishopbook.com/

[3] F.-F. Li *et al.* CS231n: Deep Learning for Computer Vision. Notas del curso. Stanford University, 2026. «Neural Networks Part 2: Setting up the Data and the Loss», apartados «Weight Initialization» y «Regularization». https://cs231n.github.io/neural-networks-2/

[4] C. Manning *et al.* CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2021. Práctica 3, «Dependency Parsing». https://web.stanford.edu/class/archive/cs/cs224n/cs224n.1214/assignments/a3.pdf

[5] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 1, «Basics», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[6] T. Hashimoto y P. Liang. CS336: Language Modeling from Scratch. Stanford University, primavera de 2026. Práctica 2, «Systems», versión 26.1.3. https://github.com/stanford-cs336/assignment2-systems

[7] X. Glorot e Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. En *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 9:249–256, 2010.

[8] D. Goldberg. What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys*, 23(1):5–48, 1991.

[9] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 4, «Numerical Computation»; cap. 7, «Regularization for Deep Learning»; cap. 8, «Optimization for Training Deep Models»; cap. 10, «Sequence Modeling: Recurrent and Recursive Nets». https://www.deeplearningbook.org/

[10] K. He, X. Zhang, S. Ren y J. Sun. Deep residual learning for image recognition. En *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, págs. 770–778, 2016. https://arxiv.org/abs/1512.03385

[11] K. He, X. Zhang, S. Ren y J. Sun. Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En *IEEE International Conference on Computer Vision (ICCV)*, 2015. https://arxiv.org/abs/1502.01852

[12] K. He, X. Zhang, S. Ren y J. Sun. Identity mappings in deep residual networks. En *European Conference on Computer Vision (ECCV)*, 2016. https://arxiv.org/abs/1603.05027

[13] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2.ª ed. SIAM, 2002.

[14] IEEE Computer Society. *IEEE Standard for Floating-Point Arithmetic*, IEEE Std 754-2019. IEEE, 2019.

[15] S. Ioffe y C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. En *International Conference on Machine Learning (ICML)*, 2015. https://arxiv.org/abs/1502.03167

[16] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh y H. Wu. Mixed precision training. En *International Conference on Learning Representations (ICLR)*, 2018. https://arxiv.org/abs/1710.03740

[17] P. Micikevicius *et al.* FP8 formats for deep learning. Prepublicación, 2022. https://arxiv.org/abs/2209.05433

[18] R. Pascanu, T. Mikolov e Y. Bengio. On the difficulty of training recurrent neural networks. En *International Conference on Machine Learning (ICML)*, PMLR 28(3):1310–1318, 2013. https://arxiv.org/abs/1211.5063

[19] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[20] B. Raj *et al.* 11-785: Introduction to Deep Learning. Carnegie Mellon University, otoño de 2025. https://deeplearning.cs.cmu.edu/

[21] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever y R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014. https://jmlr.org/papers/v15/srivastava14a.html

[22] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[24] xAI. Grok-1: código y pesos publicados. 2024. https://github.com/xai-org/grok-1

[25] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang y T.-Y. Liu. On layer normalization in the Transformer architecture. En *International Conference on Machine Learning (ICML)*, 2020. https://arxiv.org/abs/2002.04745

[26] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 4, «Linear Neural Networks for Classification»; cap. 5, «Multilayer Perceptrons»; cap. 8, «Modern Convolutional Neural Networks»; cap. 9, «Recurrent Neural Networks». https://d2l.ai/

[27] B. Zhang y R. Sennrich. Root mean square layer normalization. En *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. https://arxiv.org/abs/1910.07467